

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



MÔN HỌC: TƯ DUY TÍNH TOÁN - CSC10014
HỌC KỲ II NĂM 2026

BÁO CÁO ĐỒ ÁN

AI TOUR GUIDE

LỚP: 24CTT6 - NHÓM: 02

Giảng viên lí thuyết: Hồ Tuấn Thanh

Giảng viên thực hành: Mai Anh Tuấn

Sinh viên thực hiện: Mai Thúc Hải Đăng - 24120276
Đỗ Ngọc Hải - 24120300
Bùi Tấn Kiên - 24120349
Võ Trúc Quỳnh - 24120427
Trần Nguyên Tân - 24120438
Trần Nhật Trường - 24120486

Thành phố Hồ Chí Minh, tháng 6/2026

Mục lục

1	Group Members	4
2	Idea	4
2.1	Problem Statement	4
2.2	Product Vision	4
2.3	Target Users	5
3	Problem Analysis & Decomposition	5
3.1	Problem Analysis	6
3.1.1	Problem Elicitation & Exploration (Khơi gợi và Khám phá bài toán)	6
3.1.2	Input and Output Definition	7
3.1.3	System Constraints	7
3.2	Decomposition	8
3.2.1	Decomposition by Functional Modules (Phân rã theo chức năng)	8
3.2.2	Decomposition by Data Flow (Phân rã theo luồng dữ liệu)	9
4	System Overview	10
4.1	Main Stakeholders	10
4.2	Main Actors / User Types	10
4.3	List of Core Features	11
4.4	Explanation of Core Features (Giải thích tính năng cốt lõi)	11
4.5	Innovation Highlights	12
5	Pattern Recognition	13
5.1	Textual & Audio Pattern Recognition	13
5.2	Semantic Pattern Recognition	14
5.3	Intent & Context Pattern Recognition	14
5.4	Visual Pattern Recognition	15
6	Abstraction	15
6.1	Khái niệm và Nguyên tắc áp dụng	15
6.2	Tầng 1: Trừu tượng hóa Thực thể và Tri thức (Data Abstraction)	16
6.3	Tầng 2: Trừu tượng hóa Không gian (Spatial Abstraction)	16
6.4	Tầng 3: Trừu tượng hóa Logic và Dịch vụ (Logic & Service Abstraction)	17
6.5	Sơ đồ minh họa quá trình Trừu tượng hóa	17
7	System / Algorithm Design	18
7.1	Tổng quan Kiến trúc và Triết lý Cốt lõi	18
7.2	Thiết kế Thuật toán Retrieval-Augmented Generation (RAG)	18

7.2.1	Cơ chế Tìm kiếm Lai: Từ vựng và Ngữ nghĩa (Hybrid Search)	19
7.2.2	Tìm kiếm tích hợp Đồ thị và Mở rộng Ngữ cảnh	19
7.2.3	Chuẩn hóa Văn bản từ Giọng nói (Canonicalization)	20
7.3	Xử lý Đa phương thức và Quản lý Luồng (Unified Orchestration)	20
7.3.1	Cơ chế hoạt động của Unified Orchestrator	20
7.3.2	Pipeline Xử lý Ảnh và Xếp hạng theo Heuristic	21
7.4	Cơ chế Dự phòng (Fallback) và Đảm bảo Tính Sẵn Sàng	21
7.5	Thiết kế Cơ sở dữ liệu và Lưu trữ Ngữ cảnh	22
7.5.1	Cấu trúc Không gian, Di tích và Dữ liệu Song ngữ	22
7.5.2	Quản lý Trạng thái Hội thoại và Dữ liệu Phân tích	22
8	Implementation	23
8.1	Kiến trúc Frontend và Giao diện UI/UX	23
8.1.1	Các View chính và Bố cục Dashboard	23
8.1.2	Bản đồ Tương tác và Tích hợp GPS	23
8.1.3	Quản lý Dữ liệu Tour ở phía Client	23
8.1.4	Tích hợp Web Speech API để Đọc Văn bản Nhanh	24
8.1.5	Cài đặt Thuật toán Tìm đường và Cơ chế Chịu lỗi	24
8.1.6	Tích hợp Cổng thanh toán VNPay (Sandbox)	25
8.1.7	Hệ thống Đánh giá Địa điểm (Rating & Review)	26
8.2	Kiến trúc Backend và Xử lý Bất đồng bộ	26
8.2.1	Vòng đời Ứng dụng và Các Middleware	26
8.2.2	Giới hạn Request và Bảo mật Dữ liệu Đầu vào	27
8.2.3	Truy xuất Database Bất đồng bộ với asyncpg	27
8.3	Chiến lược Bộ nhớ đệm (Cache) và Tối ưu hóa API	27
8.3.1	Tích hợp aiocache	27
8.4	Các thách thức	28
8.4.1	Thách thức 1: Quản lý độ trễ và Lỗi gọi API từ LLM	28
8.4.2	Thách thức 2: Xử lý nhiễu âm thanh từ môi trường ngoài trời	29
8.4.3	Thách thức 3: Trải nghiệm bản đồ trên thiết bị di động yếu (Mobile Performance)	29
8.5	Tổng hợp các giải pháp triển khai	30
9	Testing	30
9.1	Unit Testing và Integration Testing (Kiểm thử Đơn vị & Tích hợp)	30
9.2	Quantitative & Qualitative Testing (Kiểm thử Định lượng & Định tính)	31
9.3	Đặc tả Các Ca kiểm thử Hệ thống (System Test Cases)	32
9.4	User Acceptance Testing (Kiểm thử Chấp nhận Người dùng)	32
10	Demo	33

10.1	Màn hình chính khi vừa truy cập web	33
10.2	Màn hình Khám phá Bản đồ	34
10.3	Màn hình Trợ lý Đa phương thức	36
10.4	Màn hình Lập lịch trình	37
10.5	Màn hình check in	38
10.6	Màn hình mua vé tham quan	39
10.7	Màn hình quiz game	40
11	Deployment	40
11.1	System Prerequisites (Yêu cầu hệ thống)	41
11.2	Execution Steps (Quy trình khởi chạy hệ thống)	41
11.3	Future Deployment Plan (Kế hoạch triển khai đám mây trong tương lai)	42
12	Logbook: Plan and Actual Workload	42
12.1	Project Management Tools (Công cụ quản lý dự án)	42
12.2	Plan Workload	43
12.3	Actual Workload	43
12.4	Workload Distribution (Bảng phân chia công việc)	44
13	AI usage and declaration	45
13.1	Tích hợp AI trong hệ thống	45
13.1.1	Chiến lược Tích hợp Trí tuệ Nhân tạo Đa luồng	45
13.1.2	Ứng dụng Google Gemini cho Xử lý Thị giác, Ngôn ngữ và Nhúng (Embedding)	45
13.1.3	Tích hợp Whisper cho Chuyển đổi Giọng nói (Speech-to-Text)	46
13.1.4	Cơ chế Dự phòng (Fallback Mechanism) với Llama-3	46
13.1.5	Kỹ nghệ Prompt (Prompt Engineering) và Quản lý Ngữ cảnh (Context)	46
13.2	DECLARATION	47
14	Conclusion and Future Work	87
14.1	Tổng kết các kết quả đạt được	87
14.2	Mặt hạn chế của hệ thống	88
14.3	Hướng phát triển trong tương lai	89
15	Phụ lục	89
15.1	Phụ lục A: Danh sách API	89
15.2	Phụ lục B: Cấu trúc Database	90
15.3	Phụ lục C: Cấu hình yêu cầu	91
15.4	Phụ lục A: Danh sách API	91

1. Group Members

MSSV	Họ và Tên	Email	Vai trò
24120276	Mai Thúc Hải Đăng	24120276@student.hcmus.edu.vn	Nhóm trưởng
24120300	Đỗ Ngọc Hải	24120300@student.hcmus.edu.vn	Thành viên
24120349	Bùi Tấn Kiên	24120349@student.hcmus.edu.vn	Thành viên
24120427	Võ Trúc Quỳnh	24120427@student.hcmus.edu.vn	Thành viên
24120438	Trần Nguyên Tân	24120438@student.hcmus.edu.vn	Thành viên
24120486	Trần Nhật Trường	24120486@student.hcmus.edu.vn	Thành viên

2. Idea

2.1. Problem Statement

[Người viết: Võ Trúc Quỳnh - 24120427]

Hiện nay, khi khách du lịch tham quan Kinh thành Huế (Đại Nội), họ thường gặp khó khăn trong việc tiếp cận thông tin chi tiết về các công trình và di tích. Các bảng thông tin tĩnh thường bị giới hạn nội dung và thiếu tính tương tác.

Mặt khác, nếu khách tham quan muốn tra cứu thông tin nhanh bằng cách chụp ảnh hiện vật hoặc tìm tuyến đường đi bộ nội khu, các ứng dụng bản đồ phổ thông không thể giải quyết được vì thiếu dữ liệu không gian đặc thù và không hiểu được ngữ cảnh di tích. Từ thực tế đó, bài toán đặt ra là cần xây dựng một hệ thống phần mềm có khả năng tương tác đa phương thức, hiểu được vị trí người dùng để làm hướng dẫn viên ảo thông minh.

2.2. Product Vision

[Người viết: Võ Trúc Quỳnh - 24120427]

Sản phẩm AI Tour Guide được định vị là một web application hướng dẫn tham quan thông minh. Mục tiêu cốt lõi của hệ thống bao gồm:

- Cho phép khách tham quan xem bản đồ di tích, chọn điểm dừng và xin chỉ đường đi bộ.
- Hỗ trợ giao tiếp và hỏi đáp thông tin thông qua nhiều phương thức: văn bản (text), hình ảnh (camera), và giọng nói (voice).

- Tích hợp AI Vision để nhận diện tự động công trình hoặc di tích trực tiếp từ ảnh chụp.
- Xử lý luồng dữ liệu thông minh: Ưu tiên trả lời dựa trên dữ liệu hiện vật (artifacts) lưu trong database trước, và chỉ sử dụng mô hình ngôn ngữ lớn (LLM) khi cần thiết diễn giải thông tin.
- Duy trì trạng thái hội thoại: Hệ thống lưu trữ ngữ cảnh hội thoại theo phiên (session) để người dùng có thể đặt câu hỏi nối tiếp một cách tự nhiên.
- Mở rộng hệ sinh thái trải nghiệm: Cung cấp thêm các tính năng như gợi ý điểm đến tiếp theo, tự động lập lịch trình (tour planning), minigame trắc nghiệm kiến thức (quiz game), hệ thống passport check-in và viết blog du lịch.

2.3. Target Users

[Người viết: Võ Trúc Quỳnh - 24120427]

- **Khách du lịch tự túc:** Những người tham quan Đại Nội Huế muốn tự do khám phá, cần công cụ chỉ đường và thuyết minh thông tin lịch sử tại chỗ.
- **Người đam mê văn hóa lịch sử:** Đối tượng cần thông tin chuẩn xác từ cơ sở dữ liệu thay vì các câu trả lời sinh ngẫu nhiên từ AI thông thường.
- **Học sinh, sinh viên:** Những người muốn ứng dụng tính năng quiz game nhiều người chơi của hệ thống để ôn tập kiến thức lịch sử ngay sau khi tham quan các di tích.

3. Problem Analysis & Decomposition

Quá trình kiến tạo và phát triển một hệ thống phần mềm trí tuệ nhân tạo có độ phức tạp cao, tích hợp hàng loạt công nghệ lõi như AI Tour Guide đòi hỏi một sự tiếp cận tư duy hệ thống nghiêm ngặt. Quá trình này bắt đầu từ việc phân tích vấn đề một cách thấu đáo (hiểu đúng và hiểu sâu những "nỗi đau" cần giải quyết của xã hội) cho đến việc áp dụng các nguyên lý phân rã hệ thống (chia nhỏ một bài toán khổng lồ thành các module có thể trị được). Dưới đây là luận giải chuyên sâu về mặt tư duy tính toán (Computational Thinking) và kỹ thuật phần mềm (Software Engineering) được áp dụng làm xương sống cho toàn bộ dự án.

3.1. Problem Analysis

[Người viết: Võ Trúc Quỳnh - 24120427]

Quá trình phân tích bài toán (Problem Analysis) giúp nhóm xác định rõ bản chất của hệ thống AI Tour Guide, từ đó định hình các ranh giới hệ thống, dữ liệu đầu vào, đầu ra và các ràng buộc kỹ thuật.

3.1.1. Problem Elicitation & Exploration (Khơi gợi và Khám phá bài toán)

[Người viết: Võ Trúc Quỳnh - 24120427]

Để hiểu rõ tại sao cần một hệ thống AI Tour Guide thay vì một ứng dụng bản đồ thông thường, nhóm áp dụng phương pháp **5 Whys** vào trải nghiệm của một du khách tại Đại Nội Huế:

- **Why 1: Tại sao du khách tự túc thường bỏ lỡ các thông tin văn hóa quan trọng?**
→ Vì khu vực Đại Nội quá rộng, bảng thông tin tại các điểm di tích lại thừa thớt và nội dung rất ngắn gọn.
- **Why 2: Tại sao họ không dùng Google tìm kiếm?**
→ Vì thao tác gõ phím tìm kiếm khi đang đi bộ rất bất tiện, và họ thường không biết chính xác tên công trình mình đang đứng trước mặt là gì để tìm.
- **Why 3: Tại sao Google Maps hay Apple Maps không giải quyết được?**
→ Vì các ứng dụng này chỉ cung cấp bản đồ đường đi cơ bản, không có sơ đồ đi bộ chi tiết nội khu và không có khả năng "thuyết minh" lịch sử.
- **Why 4: Tại sao không dùng các ứng dụng Audio Guide truyền thống?**
→ Vì Audio Guide truyền thống chỉ phát các file ghi âm có sẵn theo một kịch bản tuyến tính (1 chiều). Người dùng không thể hỏi lại nếu có thắc mắc (Ví dụ: "Vị vua này lên ngôi năm bao nhiêu?").
- **Why 5 (Cốt lõi): Tại sao chưa có giải pháp nào linh hoạt hơn?**
→ Vì việc kết hợp dữ liệu không gian (GPS, Bản đồ), kiến thức miền hẹp (Lịch sử Huế) và khả năng xử lý ngôn ngữ tự nhiên (NLP) vào cùng một hệ thống đòi hỏi một kiến trúc điều phối dữ liệu phức tạp.

Từ phân tích trên, bài toán cốt lõi được phát biểu: *"Xây dựng một hệ thống phần mềm có khả năng nhận diện ngữ cảnh không gian của người dùng, tiếp nhận đa dạng đầu*

vào (*text, voice, image*) và sử dụng cơ sở dữ liệu lịch sử để sinh ra các phản hồi hướng dẫn mang tính cá nhân hóa."

3.1.2. Input and Output Definition

[Người viết: Võ Trúc Quỳnh - 24120427]

Dưới góc độ Tư duy tính toán, hệ thống AI Tour Guide là một hàm F biến đổi tập dữ liệu đầu vào I thành tập dữ liệu đầu ra O : $F(I) = O$.

Tập dữ liệu đầu vào (Input - I):

Thành phần	Kiểu dữ liệu	Mô tả chi tiết (Ý nghĩa)
I_{text}	String	Câu hỏi dạng văn bản của người dùng (VD: "Ai xây dựng Ngọ Môn?").
I_{audio}	Audio Blob	File ghi âm giọng nói (nếu user sử dụng tính năng thu âm).
I_{image}	Base64	Ảnh chụp hiện vật từ camera để yêu cầu nhận diện.
I_{gps}	Float Tuple	Tọa độ (<i>Latitude, Longitude</i>) hiện tại của thiết bị di động.
$I_{session_mem}$	List[Dict]	Lịch sử các câu hỏi trước đó trong cùng một phiên tham quan.

Tập dữ liệu đầu ra (Output - O):

Thành phần	Kiểu dữ liệu	Mô tả chi tiết (Ý nghĩa)
O_{text}	String	Câu trả lời dạng văn bản hiển thị lên giao diện chat.
O_{speech}	String	Văn bản đã được chuẩn hóa để gửi cho engine đọc (Web Speech API).
$O_{routing}$	GeoJSON	Mảng tọa độ (Polyline) để vẽ đường đi bộ trên bản đồ Leaflet.
$O_{artifact_id}$	Integer	Mã định danh của hiện vật hệ thống nhận diện được để lưu vào bộ nhớ ngữ cảnh.

3.1.3. System Constraints

[Người viết: Võ Trúc Quỳnh - 24120427]

- **Ràng buộc về thời gian thực thi (Latency Constraint):** Vì hệ thống phục vụ du khách đang di chuyển, thời gian từ lúc gửi câu hỏi (hoặc voice) đến khi nhận được response đầu tiên phải < 3 giây. Do đó, việc gọi qua các API như Gemini hay Groq phải được xử lý bất đồng bộ (Asynchronous).
- **Ràng buộc về tính chính xác (Accuracy Constraint - No Hallucination):** Thông tin lịch sử là dữ liệu nhạy cảm. Hệ thống bị **cấm** không cho phép LLM tự do sáng tạo câu trả lời. Mọi thông tin phải được RAG (Retrieval-Augmented Generation) từ PostgreSQL trước.
- **Ràng buộc về chi phí (Cost Constraint):** API của AI (Gemini, Groq) tính phí theo token. Hệ thống cần có cơ chế Cache các câu hỏi phổ biến (như "Giờ mở cửa Đại Nội?") tại RAM hoặc Redis để tránh tốn tiền gọi API liên tục.

3.2. Decomposition

[Người viết: Võ Trúc Quỳnh - 24120427]

Hệ thống AI Tour Guide rất phức tạp, do đó nhóm áp dụng kỹ thuật Phân rã (Decomposition) để chia nhỏ bài toán lớn thành các Module (phân hệ) độc lập, giúp các thành viên trong nhóm dễ dàng chia việc và lập trình.

3.2.1. Decomposition by Functional Modules (Phân rã theo chức năng)

[Người viết: Võ Trúc Quỳnh - 24120427]

Hệ thống được chia thành 4 Sub-systems (Hệ thống con) chính:

1. Module Giao diện người dùng (Client-side UI):

- *Nhiệm vụ:* Xử lý hiển thị tương tác.
- *Thành phần nhỏ hơn:* Map Component (vẽ bản đồ, hiển thị marker), Chat Interface (quản lý tin nhắn text/voice/image), Tour Memory State (lưu giữ các điểm đã đi qua ngay trên local trình duyệt).

2. Module API Gateway & Orchestration (Lớp điều phối Backend):

- *Nhiệm vụ:* Tiếp nhận request, phân loại intent (ý định) của người dùng để gọi đúng luồng xử lý.

- *Thành phần nhỏ hơn:* Route Validator (Kiểm tra dữ liệu hợp lệ), Rate Limiter (Chống spam API), UnifiedOrchestrator (Trái tim của hệ thống, quyết định luồng chạy).

3. Module Xử lý Trí tuệ Nhân tạo (AI Core Services):

- *Nhiệm vụ:* Thực hiện các phép toán phức tạp về ngôn ngữ và thị giác.
- *Thành phần nhỏ hơn:* LLM Engine (Gọi Gemini để sinh text), Vision Engine (Phân tích ảnh hiện vật), STT Engine (Chuyển giọng nói thành chữ dùng Whisper).

4. Module Lưu trữ và Truy xuất (Data Persistence & Retrieval):

- *Nhiệm vụ:* Lưu trữ an toàn cấu trúc đồ thị di tích và lịch sử hội thoại.
- *Thành phần nhỏ hơn:* Vector Search Engine (Tìm kiếm câu hỏi tương tự bằng pgvector), Relational Tables (Quản lý Location, Artifacts, Blogs), Cache Layer.

3.2.2. Decomposition by Data Flow (Phân rã theo luồng dữ liệu)

[Người viết: Võ Trúc Quỳnh - 24120427]

Để giải quyết một yêu cầu phức tạp (Ví dụ: Người dùng vừa gửi ảnh vừa hỏi "Chỗ này xây năm nào?"), thuật toán xử lý luồng dữ liệu được phân rã thành các bước (Steps) tuần tự:

1. **Step 1 - Pre-processing (Tiền xử lý):** Nén ảnh, chuyển đổi định dạng âm thanh.
2. **Step 2 - Intent & Context Resolution (Giải quyết ngữ cảnh):** Đọc *session_memory* từ CSDL để hiểu chữ "Chỗ này" có nghĩa là gì, hoặc dùng Vision phân tích ảnh để tìm ra ID hiện vật.
3. **Step 3 - Data Retrieval (Truy xuất dữ liệu):** Thực thi câu lệnh SQL vào bảng *artifacts* và *artifact_faqs* để lấy dữ liệu năm xây dựng của hiện vật vừa xác định được ở Step 2.
4. **Step 4 - Generative Formation (Sinh câu trả lời):** Bơm dữ liệu thô lấy từ CSDL vào một Prompt Template đã thiết kế sẵn, gửi cho LLM để tạo thành một câu trả lời văn nói hoàn chỉnh.

5. **Step 5 - Post-processing (Hậu xử lý):** Lưu log vào bảng `chat_turns`, tạo thẻ TTS để trình duyệt đọc âm thanh và trả JSON về cho Frontend.

4. System Overview

4.1. Main Stakeholders

[Người viết: Võ Trúc Quỳnh - 24120427]

Dự án AI Tour Guide được xây dựng để phục vụ và giải quyết bài toán cho các nhóm lợi ích (Stakeholders) sau:

- **Khách du lịch tự túc (Tourists):** Là nhóm thụ hưởng trực tiếp giá trị của hệ thống. Họ mong muốn một công cụ không bị gò bó về thời gian như việc thuê hướng dẫn viên thực, có khả năng giải đáp thắc mắc ngay lập tức (real-time) và cung cấp thông tin lịch sử dễ hiểu, trực quan.
- **Trung tâm Bảo tồn Di tích Cố đô Huế (Ban quản lý):** Mặc dù không trực tiếp vận hành hệ thống trong giai đoạn phát triển này, nhưng họ là đơn vị sở hữu dữ liệu không gian và lịch sử. Họ quan tâm đến việc văn hóa, lịch sử không bị AI "bóp méo" (hallucination) làm sai lệch kiến thức truyền tải đến du khách.
- **Nhóm phát triển (Sinh viên thực hiện):** Chịu trách nhiệm thiết kế hệ thống, tối ưu hóa các lệnh gọi API (để giảm thiểu chi phí token của LLM) và duy trì sự ổn định của Backend FastAPI và database PostgreSQL.

4.2. Main Actors / User Types

[Người viết: Võ Trúc Quỳnh - 24120427]

Trong sơ đồ Use Case của hệ thống, chúng tôi định nghĩa 3 tác nhân (Actors) chính tương tác trực tiếp với nền tảng:

1. **Người dùng cuối (End-User):** Chủ yếu là khách tham quan sử dụng thiết bị di động (smartphone). Tác nhân này cung cấp các luồng input đa phương thức (gõ văn bản, thu âm giọng nói, chụp ảnh bằng camera, cấp quyền truy cập GPS) và nhận lại kết quả điều hướng, âm thanh thuyết minh.
2. **Hệ thống AI Ngoại vi (AI Providers - Gemini & Groq):** Đóng vai trò là các tác nhân hệ thống (System Actors). Backend sẽ giao tiếp với các tác nhân này thông qua RESTful API để tận dụng sức mạnh tính toán mà server local không kham nổi (Nhận diện hình ảnh, LLM Generation, Speech-to-Text).

3. **Hệ thống Định tuyến (Routing Engine - OSRM):** Tác nhân chuyên biệt nhận cặp tọa độ (A, B) và trả về dữ liệu hình học (GeoJSON Polyline) đường đi bộ ngắn nhất trong khuôn viên Đại Nội.

4.3. List of Core Features

[Người viết: Võ Trúc Quỳnh - 24120427]

Để đáp ứng nhu cầu tham quan thực tế, hệ thống đóng gói các tính năng cốt lõi sau:

1. Bản đồ định vị và Định tuyến đi bộ (Interactive Map & Routing).
2. Chatbot Đa phương thức (Multimodal Chat).
3. Trợ lý nhận thức ngữ cảnh (Memory-aware Orchestration).
4. Nhận diện di tích qua hình ảnh (Vision Recognition).
5. Lập lịch trình tự động (Tour Planning).
6. Ôn tập kiến thức (Quiz Game) và Sổ tay du lịch (Travel Blog).
7. Mua vé tham quan, đồ ăn và đánh giá địa điểm.

4.4. Explanation of Core Features (Giải thích tính năng cốt lõi)

[Người viết: Võ Trúc Quỳnh - 24120427]

Bảng dưới đây đặc tả chi tiết cơ chế hoạt động của từng tính năng cốt lõi đã liệt kê ở phần trên:

ID	Tính năng	Cơ chế hoạt động và Giải thích chi tiết
F01	Map & Routing	Frontend kết hợp danh sách di tích tĩnh và cấu hình từ API để render bản đồ. Gọi API OSRM để vẽ đường đi bộ trực quan.
F02	Multimodal Chat	Nhận text/audio/image. Có luồng STT (Groq Whisper) bóc băng thành text. Kết quả tự động đọc qua Web Speech API.
F03	Memory-aware	Duy trì session ngữ cảnh. Khi User hỏi bằng đại từ phiếm chỉ, hệ thống tự load context hiện tại từ PostgreSQL.
F04	Vision Recognition	Nén ảnh, gửi lên Gemini Vision lấy nhãn, map nhãn với ID di tích kết hợp heuristic khoảng cách GPS để giảm sai số.
F05	Tour Planning	Thuật toán Tham lam (Greedy) kết hợp khoảng cách Haversine tính toán thứ tự điểm đến, sau đó gọi OSRM để lấy hình học đoạn đường (leg geometry).
F06	Quiz Game	Tự động tạo câu hỏi trắc nghiệm qua Gemini dựa trên danh sách các điểm User đã check-in (visited_ids) và dữ liệu di tích.
F07	Blog & Trip Journal	Sổ tay du lịch cá nhân hóa (Trip Journal) lưu các bức ảnh (Photo Booth) và cho phép đăng bài Blog, tương tác (Like, Comment, Bookmark).
F08	Payment & Rating	Tích hợp checkout VNPAY mua vé/đồ ăn và hệ thống đánh giá địa điểm (Overall, Service, Scenery, Price) sau khi tham quan.

Bảng 1: Đặc tả chi tiết các tính năng cốt lõi của hệ thống

4.5. Innovation Highlights

[Người viết: Võ Trúc Quỳnh - 24120427]

Dự án AI Tour Guide không dừng lại ở việc thiết kế một giao diện bọc lấy API của ChatGPT. Điểm nhấn tính toán và sáng tạo kỹ thuật của nhóm nằm ở 3 khía cạnh kiến trúc sâu bên dưới:

- **DB-First, LLM-Second:** Trong domain lịch sử, việc AI sinh ra thông tin ảo là điều cấm kỵ. Nhóm thiết kế luồng Orchestrator ưu tiên việc dùng vector search (pgvector) tìm kiếm "Sự kiện lịch sử"trực tiếp từ PostgreSQL. LLM chỉ đóng vai trò "người phát ngôn"để ghép nối các dữ liệu thô này thành câu văn tự nhiên, đảm bảo độ chính xác 100% về fact.

- **Canonicalization Engine:** Nhận diện giọng nói tiếng Việt ở môi trường ngoài trời (nhiều gió, ồn) thường sinh ra các lỗi sai đồng âm (VD: STT nhận nhầm "Ngọ Môn" thành "Ngọt Môn"). Thay vì ném thẳng chữ sai này vào DB, hệ thống có một bước chạy Fuzzy Search (thuật toán `pg_trgm`) để nắn lại các thực thể bị sai chính tả thành thực thể chuẩn trước khi xử lý logic.
- **Graph-augmented Search (Tìm kiếm lân cận):** Hệ thống xây dựng một đồ thị tri thức nhỏ (bảng `artifact_relations`). Khi User hỏi về một di tích, AI không chỉ lấy thông tin của di tích đó mà còn tự động duyệt đồ thị để gợi ý các công trình lân cận, công trình cùng triều đại hoặc cùng tác giả xây dựng.

5. Pattern Recognition

Trong quá trình giải quyết bài toán cốt lõi, nhóm nhận thấy các truy vấn của du khách (dù bằng văn bản, giọng nói hay hình ảnh) đều tiềm ẩn những độ nhiễu nhất định. Kỹ thuật Nhận dạng mẫu được áp dụng ở nhiều tầng của Backend nhằm chuẩn hóa dữ liệu nhiễu này thành các thực thể mà máy tính có thể hiểu được.

5.1. Textual & Audio Pattern Recognition

[Người viết: Võ Trúc Quỳnh - 24120427]

Vấn đề: Khi người dùng sử dụng tính năng thu âm ngoài trời (nhiều tiếng ồn) hoặc gõ phím nhanh trên điện thoại, hệ thống nhận được các từ khóa sai chính tả hoặc sai dấu tiếng Việt (Ví dụ: gõ "ngõ môn", "ngọt môn" thay vì "Ngọ Môn").

Giải pháp: Áp dụng kỹ thuật so khớp mờ (Fuzzy String Matching) sử dụng module `pg_trgm` (Trigram) trong PostgreSQL và thuật toán Canonicalization tại Backend.

- **Trigram Matching:** Tách chuỗi thành các cụm 3 ký tự liên tiếp. Chuỗi "ngo" và "ngõ" sẽ có độ trùng mẫu rất thấp.
- **Unaccented Pattern:** Backend nhận dạng mẫu tiếng Việt không dấu, tự động map "dien thai hoa" thành "Điện Thái Hòa" bằng cách so sánh trên một cột `name_normalized` đã được lưu sẵn trong Database.

Nguồn nhiễu	Dữ liệu đầu vào	Mẫu nhận dạng	Chuẩn hóa
Gõ phím vội	"cug dien thai hoa"	ILIKE '%thai hoa%'	Điện Thái Hòa
STT (Whisper)	"Ngọt Môn ở đâu"	Độ tương đồng Trigram > 0.7	Ngọ Môn
Viết tắt	"đại nội"	Khớp từ điển Alias	Kinh thành Huế

Bảng 2: Bảng minh họa quy tắc nhận dạng mẫu và chuẩn hóa thực thể từ ngữ

5.2. Semantic Pattern Recognition

[Người viết: Võ Trúc Quỳnh - 24120427]

Vấn đề: Người dùng có thể hỏi cùng một ý nhưng bằng hàng chục cách diễn đạt khác nhau. Ví dụ: "Ai xây chỗ này?", "Tác giả của công trình này là ai?", "Người thiết kế là người nào?". Nếu chỉ dùng truy vấn SQL `WHERE text LIKE` truyền thống sẽ hoàn toàn thất bại.

Giải pháp: Áp dụng Nhận dạng mẫu ngữ nghĩa không gian nhiều chiều (Semantic Embeddings).

- Các "fact" lịch sử trong bảng `artifact_faqs` được chuyển đổi thành các Vector 768 chiều (sử dụng Gemini Embedding Model).
- Khi có câu hỏi mới, biến câu hỏi đó thành Vector V_q .
- Hệ thống nhận dạng mẫu bằng cách tính toán khoảng cách Cosine (Cosine Similarity) giữa V_q và các Vector trong DB bằng extension `pgvector`. Hai câu hỏi có cùng ý nghĩa (dù khác từ vựng) sẽ có vector nằm gần nhau trong không gian tọa độ.

5.3. Intent & Context Pattern Recognition

[Người viết: Võ Trúc Quỳnh - 24120427]

Đây là điểm sáng tạo cốt lõi của `UnifiedOrchestrator`. Hệ thống sử dụng LLM như một bộ phân loại (Classifier) để nhận diện mẫu "Ý định"(Intent) của người dùng dựa trên cấu trúc ngữ pháp và lịch sử hội thoại.

Có 3 mẫu Intent chính được hệ thống nhận diện và điều phối:

Mẫu Intent	Đặc điểm nhận dạng	Ví dụ Query	Hành động của Orchestrator
FOLLOW_UP	Chứa đại từ phiếm chỉ (nó, chỗ này, ông ấy). Không nêu rõ tên di tích.	"Nó được xây bằng vật liệu gì?"	Nối <code>artifact_id</code> từ câu hỏi trước, tiếp tục truy vấn DB tại di tích hiện tại.
SWITCH	Chứa tên của một di tích mới hoàn toàn khác với di tích đang xem.	"Thế còn điện Thái Hòa thì sao?"	Đổi <code>context_data</code> sang ID của Điện Thái Hòa, tìm dữ liệu mới.
SMALL_TALK	Các mẫu câu giao tiếp xã giao, chào hỏi, không chứa thực thể lịch sử.	"Chào bạn", "Cảm ơn nhé"	Bỏ qua bước gọi DB, trả lời ngay lập tức bằng Cache hoặc LLM cơ bản để tiết kiệm thời gian.

Bảng 3: Nhận dạng mẫu ý định hội thoại (Intent Classification)

5.4. Visual Pattern Recognition

[Người viết: Võ Trúc Quỳnh - 24120427]

Khi người dùng upload một bức ảnh, hệ thống không tự train một mô hình CNN (Convolutional Neural Network) từ đầu vì lượng dữ liệu ảnh của Đại Nội rất lớn và phức tạp. Thay vào đó, nhóm áp dụng phương pháp nhận dạng mẫu qua trung gian:

- Sử dụng sức mạnh của Google Gemini Vision để trích xuất các đặc trưng (features) kiến trúc từ ảnh (Ví dụ: "Cổng mái ngói màu vàng, có 3 cửa, kiến trúc cung đình").
- Trả về một tập hợp các từ khóa (Labels).
- Backend dùng tập từ khóa này so khớp ngược lại (Pattern matching) với cơ sở dữ liệu `artifacts` của hệ thống để chốt ra chính xác ID của công trình mà người dùng vừa chụp.

6. Abstraction

6.1. Khái niệm và Nguyên tắc áp dụng

[Người viết: Võ Trúc Quỳnh - 24120427]

Trong Tư duy tính toán, Trừu tượng hóa (Abstraction) là quá trình loại bỏ các chi tiết phức tạp, rườm rà của thế giới thực, chỉ giữ lại những thuộc tính cốt lõi cần thiết để máy tính có thể lưu trữ và xử lý bài toán.

Đối với dự án AI Tour Guide, một di tích như Đại Nội Huế mang rất nhiều đặc tính vật lý phức tạp (chất liệu gỗ, chiều cao, màu sắc, tình trạng thời tiết). Nếu đưa toàn bộ vào hệ thống sẽ gây quá tải dữ liệu. Nhóm đã áp dụng trừu tượng hóa ở 3 tầng kiến trúc: Trừu tượng hóa Thực thể (Entity), Trừu tượng hóa Không gian (Spatial) và Trừu tượng hóa Logic (Logic).

6.2. Tầng 1: Trừu tượng hóa Thực thể và Tri thức (Data Abstraction)

[Người viết: Võ Trúc Quỳnh - 24120427]

Các công trình kiến trúc và dữ liệu lịch sử được chuyển đổi thành cấu trúc lưu trữ cơ sở dữ liệu (Database Schema).

- **Thực thể Vật lý (Physical Artifacts):** Một công trình như Điện Thái Hòa được trừu tượng hóa thành một bản ghi (record) trong bảng **artifacts** với các thuộc tính định danh: ID, Tên, Tọa độ GPS, và ID triều đại. Các chi tiết nghệ thuật chạm trổ bị bỏ qua (hoặc chỉ lưu dưới dạng text mô tả) vì hệ thống bản đồ không cần đến chúng.
- **Tri thức Lịch sử (Historical Knowledge):** Một cuốn sách lịch sử hàng trăm trang được trừu tượng hóa thành các "Fact" ngắn. Hơn thế nữa, máy tính không hiểu ngữ nghĩa tiếng Việt, nên các Fact này được trừu tượng hóa thêm một lần nữa thành **Vector Toán học 768 chiều** (thông qua Gemini Embedding). Thay vì so sánh chữ viết, máy tính chỉ việc tính khoảng cách hình học giữa 2 vector.

6.3. Tầng 2: Trừu tượng hóa Không gian (Spatial Abstraction)

[Người viết: Võ Trúc Quỳnh - 24120427]

Để giải bài toán dẫn đường nội khu và tìm điểm đến tiếp theo, hệ thống bỏ qua các yếu tố vật lý cản trở như cây cối, mặt nước, để biến bản đồ Đại Nội thành một **Đồ thị có hướng (Directed Graph) $G = (V, E)$** .

- **Đỉnh (Vertices - V):** Trừu tượng hóa cho các điểm giao cắt đường đi hoặc vị trí các cửa, các điện thờ.
- **Cạnh (Edges - E):** Trừu tượng hóa cho các đoạn đường bộ nối giữa 2 đỉnh.
- **Trọng số (Weights):** Trừu tượng hóa cho khoảng cách vật lý (mét) tính bằng công thức Haversine hoặc thời gian đi bộ ước tính.

6.4. Tầng 3: Trừu tượng hóa Logic và Dịch vụ (Logic & Service Abstraction)

[Người viết: Võ Trúc Quỳnh - 24120427]

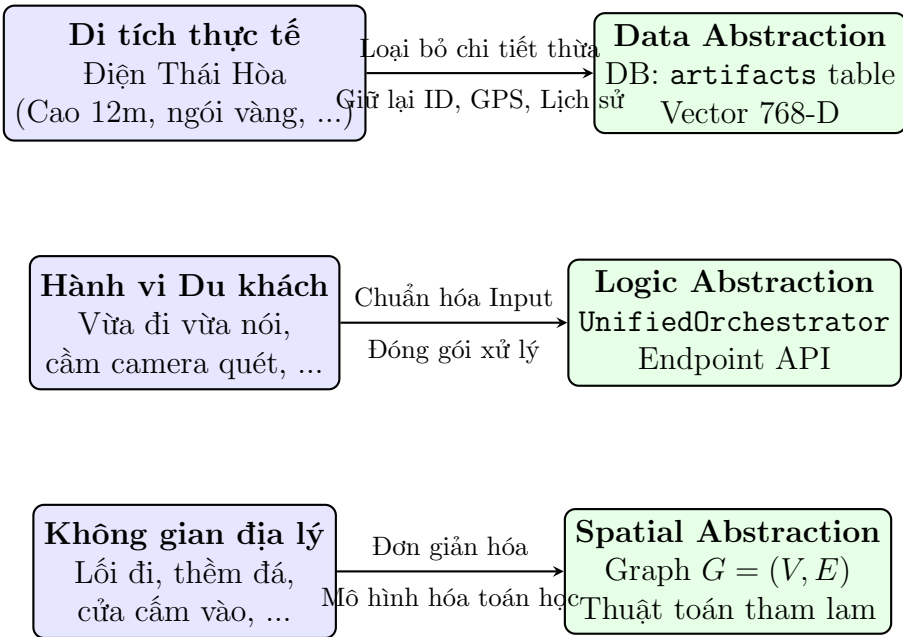
Dưới góc độ lập trình viên, việc gọi cùng lúc 3 API khác nhau (Groq để nghe, Gemini Vision để nhìn, Gemini LLM để nói) chứa rất nhiều logic phức tạp về xử lý bất đồng bộ (async), bắt lỗi mạng (timeout), và quản lý bộ nhớ.

Để giảm độ phức tạp cho các frontend developer, backend đã gom toàn bộ quá trình này và trừu tượng hóa thành một hàm duy nhất tên là `UnifiedOrchestrator.process_multimodal_request`. Frontend không cần biết bên dưới backend đang dùng AI của hãng nào, chỉ cần ném dữ liệu vào endpoint `/api/v1/chat/unified` và nhận lại chuỗi JSON kết quả.

6.5. Sơ đồ minh họa quá trình Trừu tượng hóa

[Người viết: Võ Trúc Quỳnh - 24120427]

Sơ đồ dưới đây minh họa cách hệ thống ánh xạ từ thế giới thực (Real-world Domain) sang không gian số (Digital Domain).



Hình 1: Sơ đồ minh họa sự tương quan giữa Thế giới thực và Mức độ trừu tượng hóa trong hệ thống AI Tour Guide

Khía cạnh	Thế giới thực (Phức tạp)	Hệ thống máy tính (Trừu tượng)
Phiên tham quan	Một chuyến đi dạo dài 2 tiếng, dừng nghỉ liên tục.	Một biến <code>session_id</code> và mảng <code>visited_ids</code> .
Giao tiếp của khách	Ngữ điệu, cảm xúc, tiếng ồn gió, tiếng bước chân.	Mảng byte Audio $\rightarrow$ Text transcript (Chuỗi ký tự).
Nhận diện công trình	Góc chụp xéo, bị sương mù che, ngược sáng.	File Base64 $\rightarrow$ Tập nhãn đặc trưng (Array of labels).

Bảng 4: Bảng đối chiếu các thuộc tính trước và sau khi trừu tượng hóa

7. System / Algorithm Design

7.1. Tổng quan Kiến trúc và Triết lý Cốt lõi

[Người viết: Bùi Tấn Kiên - 24120349]

Hệ thống AI Tour Guide được thiết kế theo kiến trúc đa tầng (multi-tier architecture) nhằm tối ưu hóa khả năng mở rộng, đảm bảo nhận thức ngữ cảnh và hỗ trợ tương tác đa phương thức. Thay vì xây dựng theo mô hình chatbot nguyên khối (monolithic) truyền thống, nhóm đã phân tách hệ thống thành các dịch vụ độc lập. Cụ thể, lớp Frontend được xây dựng bằng React kết hợp Vite cho hiệu năng kết xuất đồ họa cao; lớp Backend sử dụng FastAPI để xử lý I/O bất đồng bộ (asynchronous); và tầng Database sử dụng PostgreSQL để lưu trữ song song dữ liệu quan hệ và dữ liệu vector (thông qua extension `pgvector`).

Triết lý thiết kế cốt lõi của nhóm là giảm thiểu sự phụ thuộc vào các Mô hình Ngôn ngữ Lớn (LLM) ngoại vi trong việc truy xuất thông tin thực tế. Cách tiếp cận này mang lại ba lợi ích: (1) tăng tốc độ phản hồi, (2) giảm thiểu hiện tượng "ảo giác" (hallucination) thường gặp ở LLM, và (3) tối ưu hóa chi phí vận hành. Thay vì để LLM tự sinh kiến thức, hệ thống ưu tiên truy vấn cơ sở dữ liệu và duyệt Đồ thị Tri thức (Knowledge Graph) nội bộ. Sức mạnh của LLM chỉ được gọi khi thực sự cần thiết để phân tích ngữ nghĩa, tổng hợp câu trả lời tự nhiên hoặc suy luận logic phức tạp.

7.2. Thiết kế Thuật toán Retrieval-Augmented Generation (RAG)

[Người viết: Bùi Tấn Kiên - 24120349]

Nhóm đã phát triển một luồng RAG (Retrieval-Augmented Generation) đa bước, được

tùy chỉnh chuyên biệt cho miền dữ liệu lịch sử - văn hóa nhằm đảm bảo độ chính xác tuyệt đối. Cấu trúc RAG này cũng được tinh chỉnh để giải quyết các đặc thù của tiếng Việt, bao gồm vấn đề dấu câu và sự đa nghĩa theo ngữ cảnh.

7.2.1. Cơ chế Tìm kiếm Lai: Từ vựng và Ngữ nghĩa (Hybrid Search)

[Người viết: Bùi Tấn Kiên - 24120349]

Để nhận diện chính xác các di tích từ truy vấn của người dùng, hệ thống áp dụng chiến lược tìm kiếm lai. Đầu tiên là lớp tìm kiếm từ vựng (lexical search), sử dụng extension `pg_trgm` của PostgreSQL để tính toán độ tương đồng chuỗi dựa trên n-gram (trigram). Phương pháp tìm kiếm mờ (fuzzy matching) này xử lý rất tốt các lỗi đánh máy, gõ thiếu dấu hoặc lỗi nhận diện giọng nói (STT). Thuật toán thực thi qua các cấp độ: khớp chính xác (exact match), khớp mờ (`ILIKE` trigram), và cuối cùng là so khớp chuỗi không dấu.

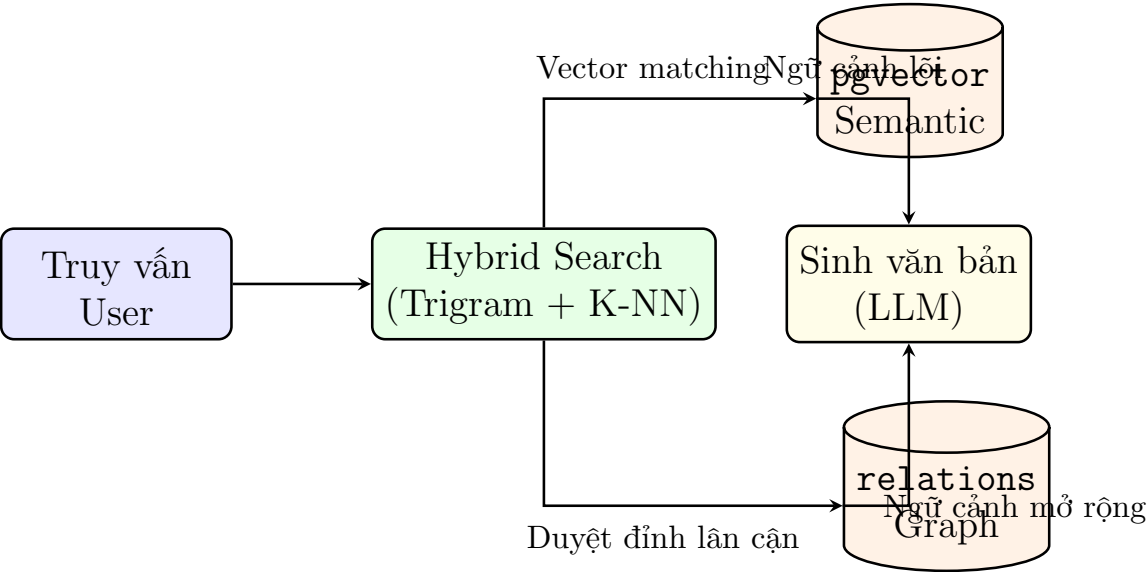
Bổ sung cho tìm kiếm từ vựng là lớp tìm kiếm ngữ nghĩa (semantic search) sử dụng `pgvector`. Bảng `artifact_faqs` chứa cột `Vector(768)` lưu trữ các embedding được trích xuất từ mô hình Gemini. Khi người dùng đặt câu hỏi mang tính ngữ nghĩa phức tạp (không chứa tên riêng định danh), hệ thống sẽ nhúng (embed) truy vấn đó thành vector 768 chiều và áp dụng thuật toán K-Nearest Neighbors (K-NN) thông qua khoảng cách Cosine để tìm kiếm tri thức tương đồng nhất.

7.2.2. Tìm kiếm tích hợp Đồ thị và Mở rộng Ngữ cảnh

[Người viết: Bùi Tấn Kiên - 24120349]

Điểm đột phá trong luồng RAG của hệ thống là sự kết hợp với Đồ thị Tri thức (Graph-Augmented Search). Các hệ thống RAG thông thường dễ đánh mất ngữ cảnh do chỉ trích xuất các phân đoạn văn bản rời rạc. Để khắc phục, nhóm xây dựng một đồ thị cục bộ thông qua bảng `artifact_relations`. Trong đó, mỗi di tích là một đỉnh (node), và các mối quan hệ như `SAME_AUTHOR` (cùng tác giả), `SAME_PERIOD` (cùng thời kỳ), hoặc `LOCATED_NEAR` (lân cận địa lý) đóng vai trò là các cạnh (edges) có trọng số.

Khi một di tích cốt lõi được xác định, thuật toán sẽ duyệt đồ thị để truy xuất thêm các di tích láng giềng cấp 1. Việc gom cụm ngữ cảnh này giúp LLM có đầy đủ dữ kiện để sinh ra các phản hồi mang tính liên kết cao (ví dụ: kết nối các nhân vật lịch sử hoặc phong cách kiến trúc), mô phỏng hoàn hảo tư duy của một hướng dẫn viên thực thụ.



Hình 2: Sơ đồ luồng RAG kết hợp tìm kiếm Lai và Đồ thị Tri thức

7.2.3. Chuẩn hóa Văn bản từ Giọng nói (Canonicalization)

[Người viết: Bùi Tấn Kiên - 24120349]

Do đặc thù các mô hình Speech-to-Text (STT) thường xuyên nhận diện sai các danh từ riêng lịch sử (ví dụ: "Ngô Môn" bị nhầm thành "ngọn mông"), hệ thống tích hợp một module chuẩn hóa trước khi đưa vào RAG. Thuật toán sẽ quét chuỗi văn bản thô, áp dụng các phép thay thế regex và so khớp từ mờ dựa trên một từ điển thực thể miền (domain dictionary) đã được định nghĩa sẵn. Việc khử nhiễu (denoise) dữ liệu đầu vào này làm tăng đáng kể độ chính xác cho quá trình tìm kiếm phía sau.

7.3. Xử lý Đa phương thức và Quản lý Luồng (Unified Orchestration)

[Người viết: Bùi Tấn Kiên - 24120349]

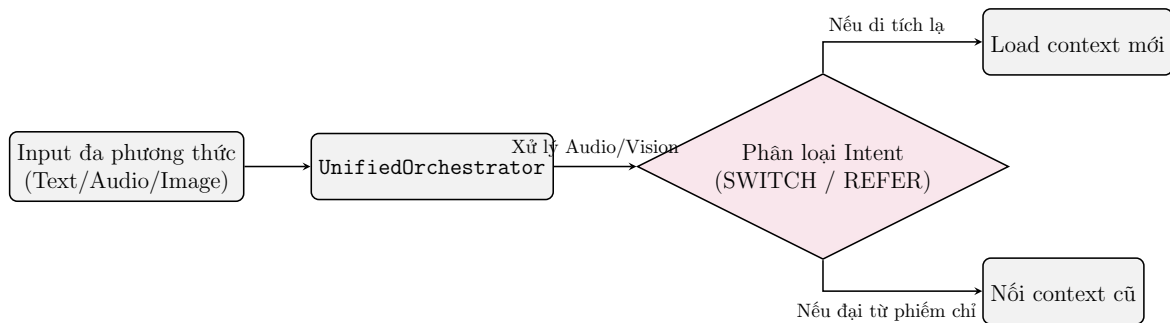
Để hệ thống có thể tiếp nhận đồng thời văn bản, âm thanh và hình ảnh một cách trơn tru, nhóm đã xây dựng UnifiedOrchestrator. Thành phần này đóng vai trò là bộ điều phối trung tâm, quản lý các tác vụ xử lý song song và máy trạng thái (state machine) của phiên hội thoại.

7.3.1. Cơ chế hoạt động của Unified Orchestrator

[Người viết: Bùi Tấn Kiên - 24120349]

Khi nhận một tải trọng (payload) đa phương thức, **UnifiedOrchestrator** sẽ kích hoạt các tiến trình bất đồng bộ. Nếu tồn tại dữ liệu âm thanh, nó được điều hướng qua pipeline STT (Groq Whisper). Đồng thời, nếu có dữ liệu hình ảnh, pipeline Vision (Gemini Vision) sẽ được kích hoạt. Sau khi hoàn tất, orchestrator mới tiến hành tổng hợp kết quả.

Tiếp theo, hệ thống đánh giá ngữ cảnh bằng cách phân tích truy vấn hiện tại, kết quả nhận diện ảnh và lịch sử hội thoại (lưu tại `chat_turns`). Thông qua một tác vụ phân loại gọn nhẹ bằng LLM, hệ thống xác định ý định của người dùng là **SWITCH** (chuyển hẳn sang chủ đề mới) hay **COMPARE_OR_REFERER** (tham chiếu/só sánh với di tích đang xem). Quá trình quản lý trạng thái này duy trì mạch hội thoại tự nhiên và liền mạch.



Hình 3: Cơ chế điều phối ngữ cảnh của Unified Orchestrator

7.3.2. Pipeline Xử lý Ảnh và Xếp hạng theo Heuristic

[Người viết: Bùi Tấn Kiên - 24120349]

Thay vì phụ thuộc hoàn toàn vào nhãn thô sinh ra từ mô hình Gemini Vision, hệ thống áp dụng cơ chế xác minh thứ cấp. Các nhãn thô được đối chiếu với danh mục đặc trưng chuẩn (`vision_artifact_features.json`). Đặc biệt, thuật toán tích hợp một hàm heuristic tính điểm phạt/thưởng dựa trên khoảng cách GPS thực tế của người dùng. Việc ràng buộc vị trí địa lý này triệt tiêu hoàn toàn sai số nhận diện giữa các công trình kiến trúc có hình dáng tương đồng trong Đại Nội.

7.4. Cơ chế Dự phòng (Fallback) và Đảm bảo Tính Sẵn Sàng

[Người viết: Bùi Tấn Kiên - 24120349]

Để tối ưu hóa chi phí và đảm bảo tính sẵn sàng cao (High Availability), nhóm đã áp dụng Design Pattern *Chain of Responsibility* thông qua lớp `FallbackLLMProvider`. Cấu hình hệ thống cung cấp tham số `LLM_PROVIDER_ORDER` để định nghĩa chuỗi ưu tiên (ví dụ: Gemini → Groq).

Mỗi provider đều bị giám sát bởi một cơ chế timeout nghiêm ngặt. Trong trường hợp API chính (Gemini) trả về lỗi 429 Too Many Requests hoặc mất kết nối, hệ thống sẽ tự động chuyển tiếp ngữ cảnh và prompt sang LLM dự phòng (Groq) mà không làm gián đoạn trải nghiệm người dùng. Việc thiết lập cơ chế này là tối quan trọng đối với ứng dụng du lịch thời gian thực. Hơn nữa, việc phân bổ nhiệm vụ rõ ràng (Gemini chuyên xử lý văn bản/hình ảnh phức tạp, Groq chuyên xử lý STT tốc độ cao và dự phòng LLM) giúp hệ thống vận hành cực kỳ hiệu quả.

7.5. Thiết kế Cơ sở dữ liệu và Lưu trữ Ngữ cảnh

[Người viết: Bùi Tấn Kiên - 24120349]

Hệ thống PostgreSQL được chuẩn hóa về Dạng chuẩn 3 (3NF) để đáp ứng các logic truy xuất phức tạp của miền du lịch.

7.5.1. Cấu trúc Không gian, Di tích và Dữ liệu Song ngữ

[Người viết: Bùi Tấn Kiên - 24120349]

Bảng `locations` quản lý các phân khu lớn (bao gồm ranh giới định vị và thời gian hoạt động). Trong khi đó, bảng `artifacts` lưu trữ dữ liệu chi tiết của từng hạng mục công trình cụ thể, bao gồm tọa độ (`latitude`, `longitude`), niên đại (`year`), và người khởi dựng (`author`). Nhằm hỗ trợ đa ngôn ngữ linh hoạt, cấu trúc nội dung được thiết kế theo mô hình đa hình (polymorphic) qua bảng `bilingual_content`. Bảng này sử dụng trường `content_type` để phân loại các mảnh thông tin (ví dụ: `visit_route`, `visit_highlights`, `photo_spots`), giúp dễ dàng mở rộng nội dung sau này mà không cần thay đổi schema.

7.5.2. Quản lý Trạng thái Hội thoại và Dữ liệu Phân tích

[Người viết: Bùi Tấn Kiên - 24120349]

Bảng `chat_turns` đóng vai trò lưu trữ bộ nhớ ngắn hạn của hệ thống, bao gồm `session_id`, `role`, văn bản gốc và một đối tượng JSON `context_data` đánh dấu ID di tích đang được thảo luận. Đây là cơ sở để Orchestrator ráp nối mạch hội thoại. Đồng thời, hệ thống thu thập các tín hiệu đánh giá thông qua bảng `feedback_events` (liên kết qua `request_id` và `message_id`). Nguồn dữ liệu này được nhóm tận dụng để đo lường độ chính xác của luồng RAG và tinh chỉnh Prompt Engineering trong các phiên bản cập nhật.

8. Implementation

8.1. Kiến trúc Frontend và Giao diện UI/UX

[Người viết: Bùi Tấn Kiên - 24120349]

Phần frontend tụi em dùng React 18 và Vite 6, code theo kiểu ưu tiên thiết bị di động (mobile-first) để phù hợp cho khách du lịch vừa đi vừa xài. Tụi em không thêm xài mấy cái thư viện định tuyến nặng nề (kiểu như React Router) mà tự viết một cái hệ thống quản lý view nhẹ gọn dựa trên state và `window.history`. Làm vậy giúp giảm dung lượng file build đi nhiều và lúc chuyển trang cũng phản hồi tức thì luôn.

8.1.1. Các View chính và Bố cục Dashboard

[Người viết: Bùi Tấn Kiên - 24120349]

Luồng chạy của app bắt đầu từ file `App.jsx`, từ đó nó chia ra hiển thị các component khác nhau. Component `HomeScreen` là màn hình chào, để người dùng chọn ngôn ngữ (Anh/Việt) và bắt đầu chuyển đi.

Giao diện chính được tụi em gom hết vào `ExploreDashboard`. Cái component bự này chứa rất nhiều hệ thống con tạo thành một trung tâm điều khiển tour. Trong này có tab `MapExplore` để xem bản đồ, `UnifiedChatPage` để chat bằng chữ/giọng/ảnh, và các nút để vào mấy tính năng phụ như buồng chụp ảnh, phòng chơi game mini và cuốn hộ chiếu du lịch kỹ thuật số của user.

8.1.2. Bản đồ Tương tác và Tích hợp GPS

[Người viết: Bùi Tấn Kiên - 24120349]

Phần bản đồ `MapExplore` sử dụng thư viện Leaflet. Danh sách các điểm di tích được hiển thị thông qua việc kết hợp 17 điểm set cứng trên Frontend và dữ liệu cấu hình gọi từ API `/api/v1/map/config`. Bên cạnh bản đồ, tụi em còn phát triển thêm module `Photo Booth` (chụp ảnh kỷ niệm) và `Trip Journal` (Sổ tay chuyến đi). Các dữ liệu này tạm thời được quản lý cục bộ thông qua `sessionStorage` để tăng tốc độ phản hồi trước khi người dùng quyết định đăng lên mạng xã hội (Blog).

8.1.3. Quản lý Dữ liệu Tour ở phía Client

[Người viết: Đỗ Ngọc Hải - 24120300]

Một trong những thành phần quan trọng của hệ thống là tệp `tourMemoryService.js`, đảm nhiệm vai trò quản lý trạng thái (state) cho tính năng "Hộ chiếu Du lịch" của người dùng. Nhằm ngăn ngừa tình trạng mất dữ liệu khi tải lại trang hoặc trình duyệt gặp sự cố, nhóm đã thiết lập cơ chế lưu trữ toàn bộ trạng thái tour vào `sessionStorage` của trình duyệt. Dữ liệu được bảo lưu bao gồm `sessionId`, thời gian `check-ins` tại các điểm di tích, hình ảnh `photos` đã chụp, các câu hỏi `questions` đã đặt ra cho AI, và lịch sử di chuyển `route history`. Phần vùng bộ nhớ này ở phía Client hoạt động hoàn toàn độc lập với bảng `chat_turns` dưới Backend, tập trung phục vụ các tính năng tương tác giải trí (gamification) và lưu trữ kỷ niệm chuyến đi của người dùng.

8.1.4. Tích hợp Web Speech API để Đọc Văn bản Nhanh

[Người viết: Đỗ Ngọc Hải - 24120300]

Mặc dù Backend đã được cấu hình dịch vụ đọc văn bản Edge TTS, nhóm quyết định triển khai trực tiếp Web Speech API (`SpeechSynthesisUtterance`) tích hợp sẵn trên trình duyệt cho phía Frontend. Lựa chọn này giúp triệt tiêu độ trễ mạng khi tải các tệp âm thanh dung lượng lớn, từ đó tối ưu hóa tốc độ phản hồi của trợ lý ảo. Frontend cũng được trang bị một bộ quản lý hàng đợi (queue) TTS chuyên dụng, có chức năng phân tách các câu trả lời dài của LLM thành những đoạn ngắn tự nhiên. Cơ chế này cho phép người dùng linh hoạt điều khiển luồng âm thanh (phát, tạm dừng, nghe tiếp, hoặc tắt), đồng thời hệ thống sẽ tự động lựa chọn giọng đọc phù hợp với ngôn ngữ đang sử dụng (`vi-VN` hoặc `en-US`).

8.1.5. Cài đặt Thuật toán Tìm đường và Cơ chế Chịu lỗi

[Người viết: Đỗ Ngọc Hải - 24120300]

Tính năng định tuyến trên bản đồ được thiết kế với cơ chế chịu lỗi (Fault Tolerance) cao. Khi gọi API `/api/v1/map/route`, `OsrmService` sẽ ưu tiên gửi yêu cầu đến máy chủ định tuyến công cộng OSRM (Open Source Routing Machine). Trong trường hợp máy chủ OSRM không phản hồi hoặc trả về dữ liệu lỗi, hệ thống sẽ tự động kích hoạt cơ chế dự phòng (Fallback) sử dụng thuật toán tính toán cục bộ. Phân hệ dự phòng này ứng dụng thư viện `NetworkX` để phân tích tệp đồ thị tĩnh tại `backend/data/map_graph.json` dựa trên thuật toán Dijkstra. Cơ chế này đảm bảo tính năng chỉ đường luôn hoạt động ổn định kể cả khi mất kết nối đến các dịch vụ bên ngoài.

8.1.6. Tích hợp Cổng thanh toán VNPay (Sandbox)

[Người viết: Đỗ Ngọc Hải - 24120300]

Nhận thấy du khách không chỉ có nhu cầu tra cứu thông tin mà còn cần mua vé tham quan hoặc đặt trước các dịch vụ đi kèm (như ẩm thực, nước uống), nhóm đã tích hợp cổng thanh toán điện tử VNPay vào hệ thống để tạo ra một chu trình trải nghiệm khép kín.

- **Xử lý tại Backend (FastAPI):** Nhóm đã phát triển module `vnpay_service.py` để xử lý logic tạo URL thanh toán. Điểm cốt lõi về bảo mật nằm ở thuật toán mã hóa **HMAC-SHA512**. Toàn bộ thông tin đơn hàng (số tiền, mã đơn `vnp_TxnRef`, nội dung) đều được băm (hash) cùng với một Khóa bí mật (Secret Key) để tạo ra chữ ký `vnp_SecureHash`. Việc này giúp ngăn chặn các hành vi can thiệp, chỉnh sửa dữ liệu trên đường truyền. Khi VNPay phản hồi (Return URL), backend tiếp tục băm các tham số trả về để đối chiếu (verify), đảm bảo tính toàn vẹn trước khi cập nhật trạng thái đơn hàng vào Database.
- **Xử lý Giao diện Frontend (React):** Để tối ưu hóa trải nghiệm người dùng, nhóm đã thiết kế giao diện thanh toán (`DienLongAnCheckout.jsx`, `NgocMonCheckout.jsx`) và khu vực Đánh giá (Rating Section) theo phong cách hiện đại, trực quan. Đặc biệt, người dùng không cần phải vào trang chi tiết của từng địa điểm mà có thể chọn nút "Mua vé tham quan" (kèm popup chọn loại vé) trực tiếp ngay trên trang chủ (`HomeScreen.jsx`) để thao tác nhanh gọn hơn.
- **Xử lý thanh toán qua cửa sổ Popup (Iframe Handling):** Do ứng dụng được thiết kế để có thể nhúng dưới dạng iframe (giả lập giao diện điện thoại trên web), việc chuyển trang (redirect) trực tiếp qua cổng VNPay dễ gây ra lỗi hiển thị. Vì vậy, nhóm đã xây dựng luồng thanh toán thông qua việc mở cửa sổ mới (Popup). Khi giao dịch hoàn tất trên VNPay, component `PaymentResult.jsx` sẽ tự động truyền tín hiệu xác nhận thành công về cửa sổ gốc (opener) và đóng popup lại, tạo ra trải nghiệm thanh toán liền mạch.
- **Phương án dự phòng MockVnPay (Fallback):** Nhằm đối phó với tình huống trình duyệt của người dùng bật chế độ chặn cửa sổ bật lên (block popups), nhóm đã chủ động lập trình thêm component giả lập `MockVnPay`. Component này hoạt động như một luồng thanh toán dự phòng nội bộ, đảm bảo quy trình mua vé của du khách luôn thông suốt và không bị kẹt trong mọi tình huống kỹ thuật.

8.1.7. Hệ thống Đánh giá Địa điểm (Rating & Review)

[Người viết: Đỗ Ngọc Hải - 24120300]

Nhằm đo lường mức độ hài lòng của du khách, nhóm đã phát triển thêm tính năng đánh giá trải nghiệm trực tiếp trên ứng dụng.

- **Cơ sở dữ liệu:** Các đánh giá được lưu trữ tại bảng `location_ratings`, bao gồm thông tin ID người dùng, ID địa điểm, số điểm đánh giá (1 đến 5 sao) và nội dung bình luận.
- **Trải nghiệm UI/UX:** Nhóm tích hợp các component `StarRating.jsx` và `RatingSection.jsx` giúp thao tác đánh giá trở nên trực quan. Khối lượng dữ liệu thu thập được trong tương lai sẽ làm nền tảng cho thuật toán Gợi ý (Recommendation), giúp ưu tiên hiển thị các địa điểm được đánh giá cao lên đầu lộ trình cho những du khách khác.

8.2. Kiến trúc Backend và Xử lý Bất đồng bộ

[Người viết: Đỗ Ngọc Hải - 24120300]

Về Backend, nhóm áp dụng kiến trúc nguyên khối (monolithic) dựa trên framework FastAPI, tối đa hóa khả năng xử lý đồng thời và hạn chế hiện tượng nghẽn I/O. FastAPI được lựa chọn nhờ khả năng khai thác tối đa thư viện `asyncio` của Python, giúp hệ thống có thể xử lý song song các tác vụ gọi API mạng (đến LLM) và truy vấn Database, đáp ứng tốt kịch bản lưu lượng truy cập cao.

8.2.1. Vòng đời Ứng dụng và Các Middleware

[Người viết: Đỗ Ngọc Hải - 24120300]

Tệp `main.py` đóng vai trò khởi tạo vòng đời ứng dụng và liên kết các router con (`vision_router`, `voice_router`, `chat_router`, `map_router`, `game_router`, `blog_router`). Khi khởi động, ứng dụng thiết lập hệ thống ghi log, khởi tạo kết nối Database qua hàm `create_async_engine` của SQLAlchemy và nạp sẵn các lớp cache.

Nhóm cũng thiết lập một lớp Middleware bảo mật cho mọi request đầu vào. Middleware chính sẽ đính kèm một `request_id` (UUID) nhằm phục vụ công tác truy vết lỗi (tracing). Đồng thời, nó đo lường thời gian thực thi, thống kê lưu lượng và ghi log theo định dạng chuẩn. Các số liệu giám sát này có thể được truy xuất thông qua API `/api/v1/metrics`.

8.2.2. Giới hạn Request và Bảo mật Dữ liệu Đầu vào

[Người viết: Đỗ Ngọc Hải - 24120300]

Để ngăn chặn các cuộc tấn công rác (spam) và kiểm soát chi phí sử dụng LLM API, nhóm đã tích hợp thư viện `SlowAPI`, giới hạn mặc định ở mức 30 requests/phút trên mỗi IP. Bên cạnh đó, module `request_validation.py` được thiết lập để kiểm duyệt chặt chẽ tính hợp lệ (Payload Validation) ngay từ tầng Gateway. Hệ thống sẽ tự động cắt giảm các chuỗi văn bản quá giới hạn và từ chối các tệp hình ảnh/âm thanh vượt mức dung lượng cho phép. Khi phát hiện vi phạm, API lập tức trả về lỗi HTTP 413 Payload Too Large hoặc HTTP 422 Unprocessable Entity trước khi chuyển đến AI Core, giúp tiết kiệm tối đa tài nguyên máy chủ.

8.2.3. Truy xuất Database Bất đồng bộ với `asyncpg`

[Người viết: Đỗ Ngọc Hải - 24120300]

Toàn bộ quá trình tương tác cơ sở dữ liệu đều được cấu hình bất đồng bộ thông qua driver `asyncpg` dưới nền `SQLAlchemy 2.0`. Tệp `core/database/__init__.py` sử dụng `async_sessionmaker` để khởi tạo các phiên giao dịch (session) độc lập cho từng request. Tệp `ArtifactRepository` đóng gói các thao tác ORM và Core phức tạp. Chẳng hạn, tại bước tìm kiếm mờ (Fuzzy Search), nhóm kết hợp lệnh SQL thuần để gọi toán tử `%>` của `pg_trgm` và `<=>` của `pgvector`. Phương pháp này dung hòa được sự tiện dụng của ORM đồng thời đẩy hiệu suất truy xuất lên mức tối ưu.

8.3. Chiến lược Bộ nhớ đệm (Cache) và Tối ưu hóa API

[Người viết: Đỗ Ngọc Hải - 24120300]

Nhằm giảm tải cho Database và các LLM API, nhóm triển khai cơ chế bộ nhớ đệm ở nhiều lớp kiến trúc.

8.3.1. Tích hợp `aiocache`

[Người viết: Đỗ Ngọc Hải - 24120300]

Tệp `backend/core/cache.py` đóng gói một giao diện cache bất đồng bộ sử dụng thư viện `aiocache`. Tùy thuộc vào biến môi trường, hệ thống sẽ tự động khởi tạo `SimpleMemoryCache` (cho máy chủ đơn) hoặc `RedisCache` (cho kịch bản phân tán). Lớp cache này được ứng dụng mạnh mẽ tại `ResponseGenerator` thuộc phân hệ nhận diện hình ảnh, nhằm lưu trữ các đoạn văn thuyết minh sinh động với khóa (key) kết

hợp từ `artifact_id`, `language` và `question`. Ngoài ra, `UnifiedOrchestrator` cũng khai thác bộ nhớ RAM để trả lời tức thì các câu hỏi trùng lặp hoặc các câu giao tiếp cơ bản (small talk).

8.4. Các thách thức

[Người viết: Đỗ Ngọc Hải - 24120300]

Quá trình hiện thực hóa hệ thống từ bản vẽ thiết kế thành một ứng dụng web hoàn chỉnh gặp không ít khó khăn về mặt giới hạn phần cứng, mạng và độ trễ của AI. Dưới đây là các thách thức kỹ thuật lớn nhất và cách nhóm đã giải quyết.

8.4.1. Thách thức 1: Quản lý độ trễ và Lỗi gọi API từ LLM

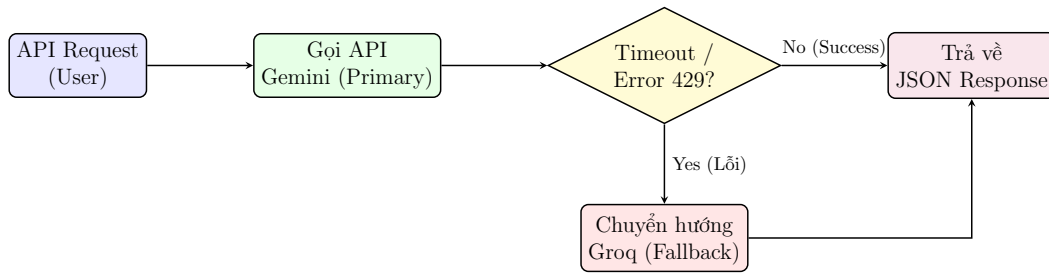
[Người viết: Đỗ Ngọc Hải - 24120300]

Vấn đề: Hệ thống phụ thuộc vào API của bên thứ 3 (Google Gemini và Groq). Trong quá trình test, nhóm phát hiện:

- API Gemini đôi khi phản hồi rất chậm ($> 10s$) vào giờ cao điểm, gây ra lỗi `ReadTimeout`.
- Nếu nhiều user cùng chat một lúc, hệ thống dễ bị Google chặn do vượt quá giới hạn lượt gọi (HTTP 429 Too Many Requests).
- Trong môi trường FastAPI (single-thread event loop), nếu sử dụng thư viện request đồng bộ (`requests`), toàn bộ server sẽ bị block khi chờ AI trả lời.

Giải pháp:

1. **Cơ chế Bất đồng bộ (Asynchronous I/O):** Thay thế thư viện `requests` bằng `httpx` và sử dụng `async/await` ở toàn bộ các endpoint. Nhờ đó, FastAPI có thể phục vụ hàng ngàn request khác trong lúc chờ LLM trả lời.
2. **Giới hạn truy cập (Rate Limiting):** Tích hợp thư viện `SlowAPI` tại lớp Middleware để giới hạn mỗi IP chỉ được gửi tối đa 30 requests/phút, bảo vệ quota của API Key.
3. **Cơ chế Dự phòng (Fallback Provider):** Xây dựng lớp `FallbackLLMProvider`. Nếu Gemini trả về lỗi hoặc timeout sau 5 giây, khối `try...catch` sẽ tự động chuyển hướng request sang Groq (chạy mô hình Llama 3) để đảm bảo user luôn nhận được câu trả lời.



Hình 4: Cơ chế FallbackLLMProvider xử lý sự cố API bên thứ 3

8.4.2. Thách thức 2: Xử lý nhiều âm thanh từ môi trường ngoài trời

[Người viết: Đỗ Ngọc Hải - 24120300]

Vấn đề: Đặc thù của ứng dụng là sử dụng ngoài trời (Đại Nội Huế). Tạp âm từ gió, tiếng bước chân và tiếng đám đông khiến mô hình Groq Whisper (Speech-to-Text) nhận diện sai các danh xưng lịch sử cổ (Ví dụ: "Cửu Đỉnh" bị nghe thành "Cựu đỉnh", "Thái Hòa" thành "Thái hỏa").

Giải pháp: Thay vì bắt user thu âm lại, nhóm thiết kế một **Pipeline Canonicalization (Chuẩn hóa)** chạy ngay sau STT:

- **Bước 1: Lọc nhiễu (Regex/Stopwords):** Loại bỏ các từ ngập ngừng (ờ, à, ừm) mà STT sinh ra.
- **Bước 2: Fuzzy Map với Database:** Sử dụng hàm `word_similarity` của `pg_trgm` để so khớp các danh từ trong câu nói với cột `name_vi` trong bảng `artifacts`. Các cụm từ sai lệch dưới 30% sẽ tự động bị thay thế bằng tên gốc trong cơ sở dữ liệu.

8.4.3. Thách thức 3: Trải nghiệm bản đồ trên thiết bị di động yếu (Mobile Performance)

[Người viết: Đỗ Ngọc Hải - 24120300]

Vấn đề: Thuật toán tìm đường (Routing) trên đồ thị $G = (V, E)$ nếu đem thực thi tại Frontend (bằng Javascript) sẽ làm trình duyệt của các dòng điện thoại cũ bị đơ (freeze) do file đồ thị lớn và quá trình duyệt mảng tốn nhiều CPU.

Giải pháp:

- Áp dụng triết lý **"Thin Client, Fat Server"**. Toàn bộ thuật toán Greedy Plan Tour và Graph Routing được chuyển về chạy trên Backend FastAPI (sử dụng thư

viện `NetworkX` của Python để tính toán đồ thị).

- Frontend chỉ đóng vai trò nhận một cục dữ liệu tọa độ (GeoJSON Polyline) siêu nhẹ và sử dụng `React Leaflet` để vẽ đè (overlay) lên bản đồ, đảm bảo khung hình (FPS) luôn đạt 60fps trên mobile.

8.5. Tổng hợp các giải pháp triển khai

[Người viết: Mai Thúc Hải Đăng - 24120276]

Bảng dưới đây tóm tắt lại các kỹ thuật tối ưu hóa đã được áp dụng trong giai đoạn lập trình thực tế:

Thành phần	Khó khăn gặp phải	Giải pháp kỹ thuật
Text-to-Speech (TTS)	Tải file MP3 sinh từ API về máy tốn băng thông và tốn thời gian tải > 2s.	Dùng Web Speech API có sẵn của trình duyệt. Không cần tải file, đọc ngay lập tức.
Image Upload (Vision)	Ảnh từ điện thoại thường > 5MB, gửi lên Backend rất lâu, gây Timeout.	Frontend dùng HTML5 Canvas nén ảnh (Compress) xuống định dạng WebP < 500KB trước khi gọi API.
Database Concurrency	Hàm <code>session.commit()</code> của SQLAlchemy truyền thống làm block các request khác.	Chuyển sang <code>SQLAlchemy Async</code> kết hợp driver <code>asyncpg</code> để truy vấn DB phi đồng bộ.

Bảng 5: Các cải tiến về mặt triển khai kỹ thuật (Implementation Improvements)

9. Testing

Quá trình kiểm thử của hệ thống `AI Tour Guide` được xây dựng dựa trên phương pháp luận kết hợp: từ kiểm thử mức thấp (Unit Testing) để xác thực logic thuật toán và các service, đến kiểm thử hệ thống tổng thể thông qua các đánh giá định lượng và định tính.

9.1. Unit Testing và Integration Testing (Kiểm thử Đơn vị & Tích hợp)

[Người viết: Mai Thúc Hải Đăng - 24120276]

Thay vì chỉ kiểm tra các hàm thuật toán đơn lẻ, nhóm đã xây dựng một hệ thống test script thực tế bao phủ cả Frontend và Backend, giúp phát hiện lỗi sớm trong quá trình phát triển.

- **Backend Testing (Sử dụng pytest):** Nhóm đã viết các file test chuyên biệt cho từng phân hệ logic cốt lõi. Cụ thể:
 - `test_unified_chatbot.py`: Kiểm tra luồng xử lý đa phương thức, quản lý ngữ cảnh (context switch/compare) và dự phòng LLM (fallback).
 - `test_voice_features.py`: Xác thực khả năng bóc băng và chuẩn hóa (canonicalization) thực thể lịch sử từ giọng nói.
 - `test_tour_service.py` & `test_game_service.py`: Kiểm thử tính đúng đắn của thuật toán tạo lộ trình và luồng logic tạo phòng/trả lời câu hỏi của Quiz Game.
 - `test_rag_database.py` & `test_api_contract.py`: Kiểm tra khả năng truy xuất Vector DB và tính toàn vẹn của các endpoint.
- **Frontend Testing:** Nhóm triển khai các script test như `webSpeechTts.test.mjs` để kiểm tra hàng đợi đọc văn bản tự động, và `photoBoothAlbum.test.mjs` để xác thực luồng lưu trữ ảnh cục bộ.

9.2. Quantitative & Qualitative Testing (Kiểm thử Định lượng & Định tính)

[Người viết: Mai Thúc Hải Đăng - 24120276]

Do giới hạn về tài nguyên triển khai, các bài kiểm thử chịu tải (Load Test) tự động chuyên sâu chưa được đính kèm vào repository. Thay vào đó, nhóm thực hiện đo lường định lượng thông qua kiểm thử thủ công và ghi nhận các chỉ số từ hệ thống giám sát (`/api/v1/metrics`):

- **Độ trễ (Latency):** Tốc độ bóc băng giọng nói (STT Whisper) thông qua chip LPU của Groq đạt hiệu năng xuất sắc (trung bình dưới 0.8 giây). Tuy nhiên, độ trễ API Text Chat dao động mạnh (2.5 – 4.5 giây) do phụ thuộc vào tình trạng mạng của Google Gemini, củng cố sự cần thiết của cơ chế `FallbackLLMProvider`.
- **Tỷ lệ nhận diện & Ảo giác:** Thông qua các kịch bản test hỏi đáp trực tiếp (Direct DB Answer), cơ chế DB-first giúp hệ thống giảm thiểu đáng kể tình trạng hallucination so với việc sử dụng LLM thuần túy. Nhờ module `pg_trgm`, tỷ lệ nắn chỉnh lỗi chính tả từ âm thanh đạt mức cao.

Về mặt định tính, giao diện được đánh giá thân thiện trên thiết bị di động, giọng đọc Web Speech API được điều chỉnh tham số `rate` và `pitch` phù hợp với văn phong thuyết minh du lịch.

9.3. Đặc tả Các Ca kiểm thử Hệ thống (System Test Cases)

[Người viết: Mai Thúc Hải Đăng - 24120276]

Các ca kiểm thử hệ thống được thiết kế theo luồng tương tác thực tế nhằm xác minh sự kết nối hoàn hảo giữa Frontend, Backend và AI Core.

TC ID	Luồng chức năng	Kịch bản Kiểm thử (Scenario)	Kết quả thực tế
TC01	Memory & Intent	Input: Đang xem Ngọ Môn. Gõ: "Nó được xây năm nào?" Kỳ vọng: AI hiểu "Nó" là Ngọ Môn và trả lời năm 1833.	Hệ thống trả lời đúng ngữ cảnh (1833).
TC02	Context Switch	Input: Đang xem Ngọ Môn. Gõ: "Thế còn Cửu Đỉnh?" Kỳ vọng: Nhận diện Intent SWITCH, cập nhật dữ liệu Cửu Đỉnh.	Backend trả về Context mới. Frontend tiếp nhận và có thể cập nhật active artifact hoặc chỉ đường tương ứng.
TC03	AI Hallucination Prevention	Input: Hỏi: "Ai là người thiết kế tháp Eiffel?" Kỳ vọng: Hệ thống từ chối trả lời do ngoài phạm vi dữ liệu Đại Nội.	AI từ chối trả lời, ưu tiên tính xác thực của Database.
TC04	Vision Recognition	Input: Chụp một góc nghiêng khuất sáng của Hiển Lâm Các. Kỳ vọng: Nhận diện đúng Hiển Lâm Các.	AI nhận diện sai, nhưng bộ lọc Heuristic/GPS đã giúp giảm thiểu sai số bằng cách cảnh báo vị trí không khớp.

Bảng 6: Đặc tả ca kiểm thử hệ thống (System Testing)

9.4. User Acceptance Testing (Kiểm thử Chấp nhận Người dùng)

[Người viết: Mai Thúc Hải Đăng - 24120276]

Nhóm tiến hành một kế hoạch kiểm thử thực nghiệm thu nhỏ thông qua giả lập định vị (GPS Mocking) trên trình duyệt để đánh giá trải nghiệm thực tế:

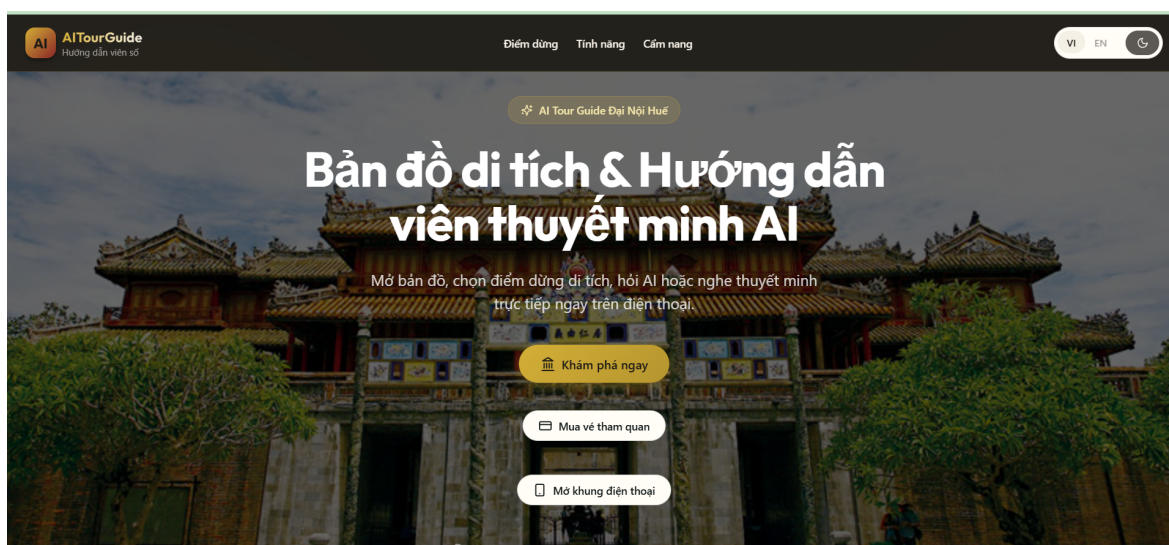
- **Phản hồi luồng điều hướng:** Việc tích hợp bản đồ Leaflet hoạt động trơn tru. Tuy nhiên, luồng tạo lộ trình (Plan Tour) đôi khi thiếu định hướng phương vị (Compass) trên trình duyệt máy tính, đòi hỏi người dùng phải tự điều chỉnh hướng nhìn.
- **Phản hồi tương tác Đa phương thức:** Tính năng thu âm và chụp ảnh phản hồi mượt mà. Việc chuyển đổi từ TTS server-side sang Web Speech API của Frontend đã giải quyết triệt để vấn đề chờ tải file âm thanh nặng, giúp cuộc hội thoại diễn ra tự nhiên và liền mạch hơn.

10. Demo

Hệ thống AI Tour Guide được thiết kế ưu tiên cho giao diện thiết bị di động (Mobile-First Design) vì người dùng sẽ thao tác trực tiếp khi đang đi bộ tham quan ngoài trời. Dưới đây là các màn hình chức năng quan trọng nhất của hệ thống.

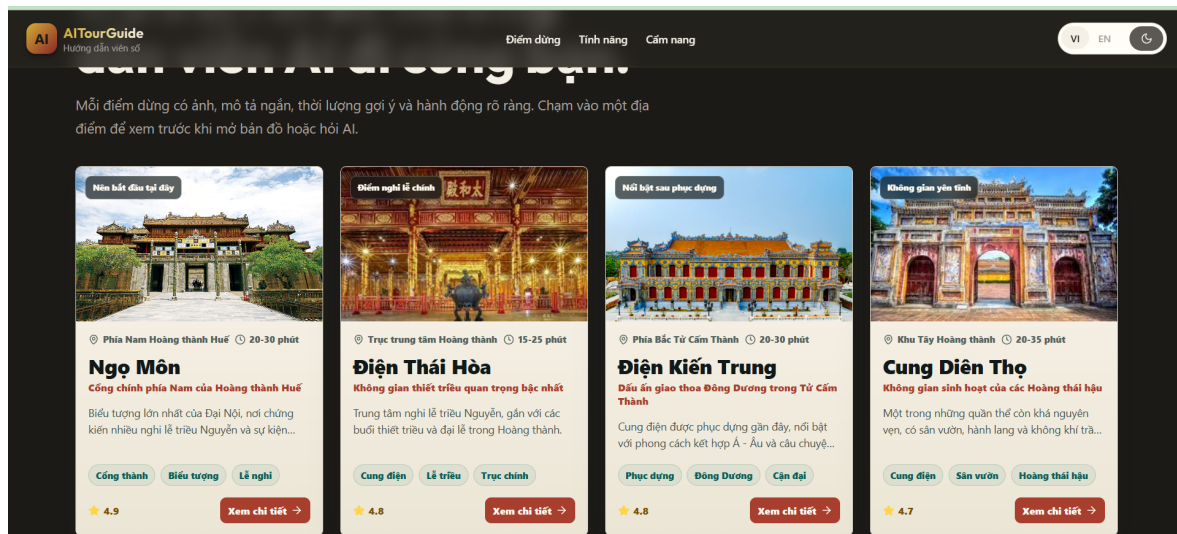
10.1. Màn hình chính khi vừa truy cập web

Khi truy cập trang web sẽ có 3 chế độ ở trang đầu tiên để lựa chọn bao gồm: Khám phá ngay để trải nghiệm dạng web trên PC, Mua vé tham quan để mua vé online và Mở khung điện thoại để được trải nghiệm như trên điện thoại di động.



Hình 5: Giao diện chính khi truy cập vào web

Ở màn hình chính cũng bao gồm 17 địa điểm tham quan Đại nội Huế để người dùng có thể tìm hiểu sơ lược về nơi đó trước khi vào tham quan.

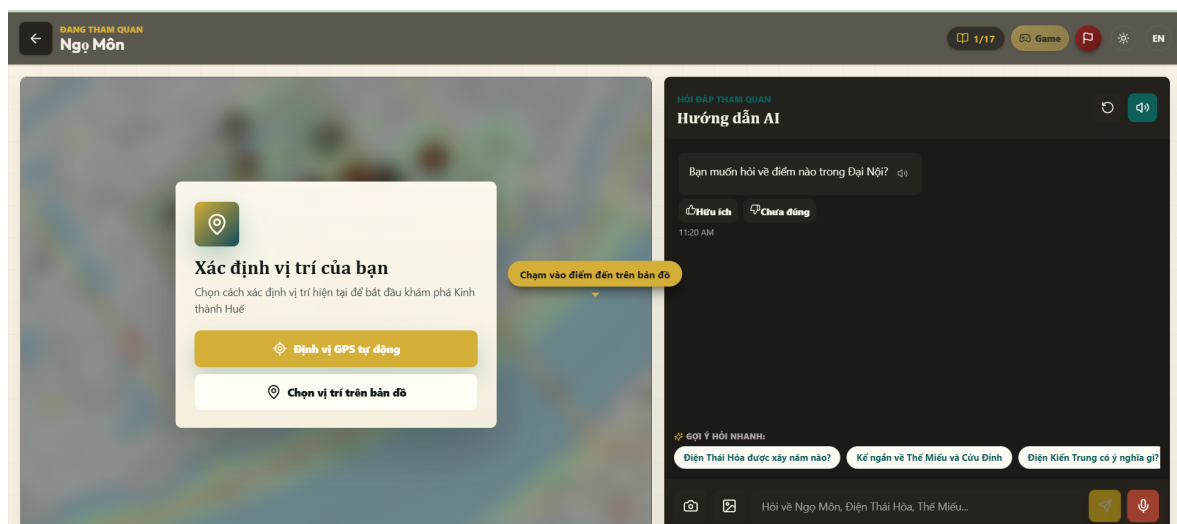


Hình 6: Giao diện chính khi truy cập vào web

10.2. Màn hình Khám phá Bản đồ

[Người viết: Mai Thúc Hải Đăng - 24120276]

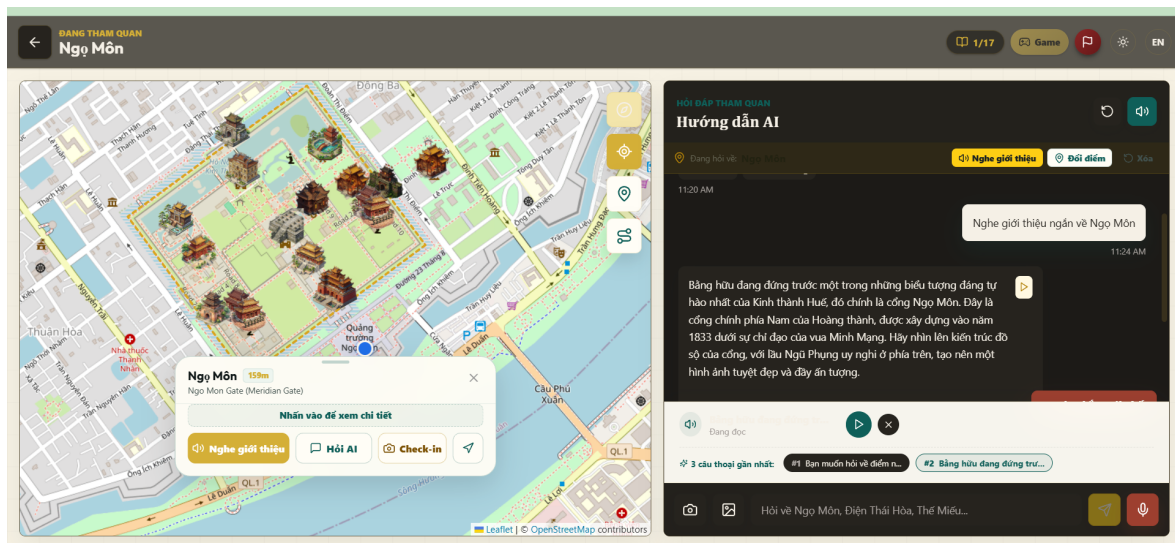
Đây là màn hình xuất hiện khi người dùng lựa chọn Khám phá ngay. Màn hình kết hợp giữa việc định vị GPS theo thời gian thực và hiển thị các điểm di tích trong Đại Nội.



Hình 7: Giao diện định vị bản đồ

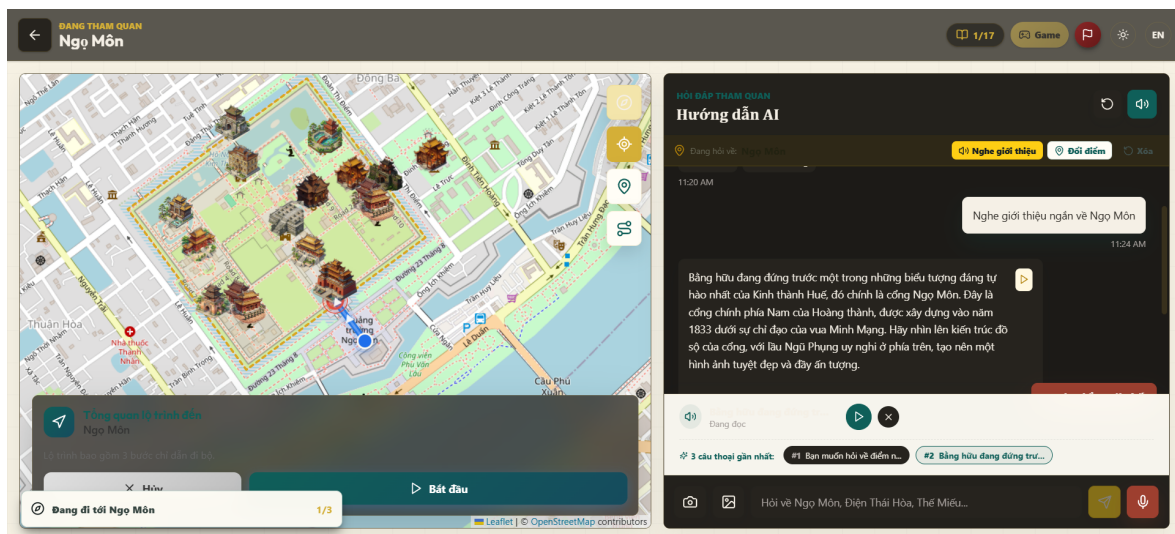
Khi đã xác định được vị trí trên bản đồ, người dùng có thể chọn vào các icon di tích

trên bản đồ để trải nghiệm các chức năng như: nghe giới thiệu, hỏi AI, check in hoặc dẫn đường từ vị trí hiện tại đến đó.



Hình 8: Giao diện định vị bản đồ

Khi chọn vào mũi tên dẫn đường giao diện sẽ giống như khi ta dùng google map.



Hình 9: Giao diện định vị bản đồ

Giải thích UI/UX:

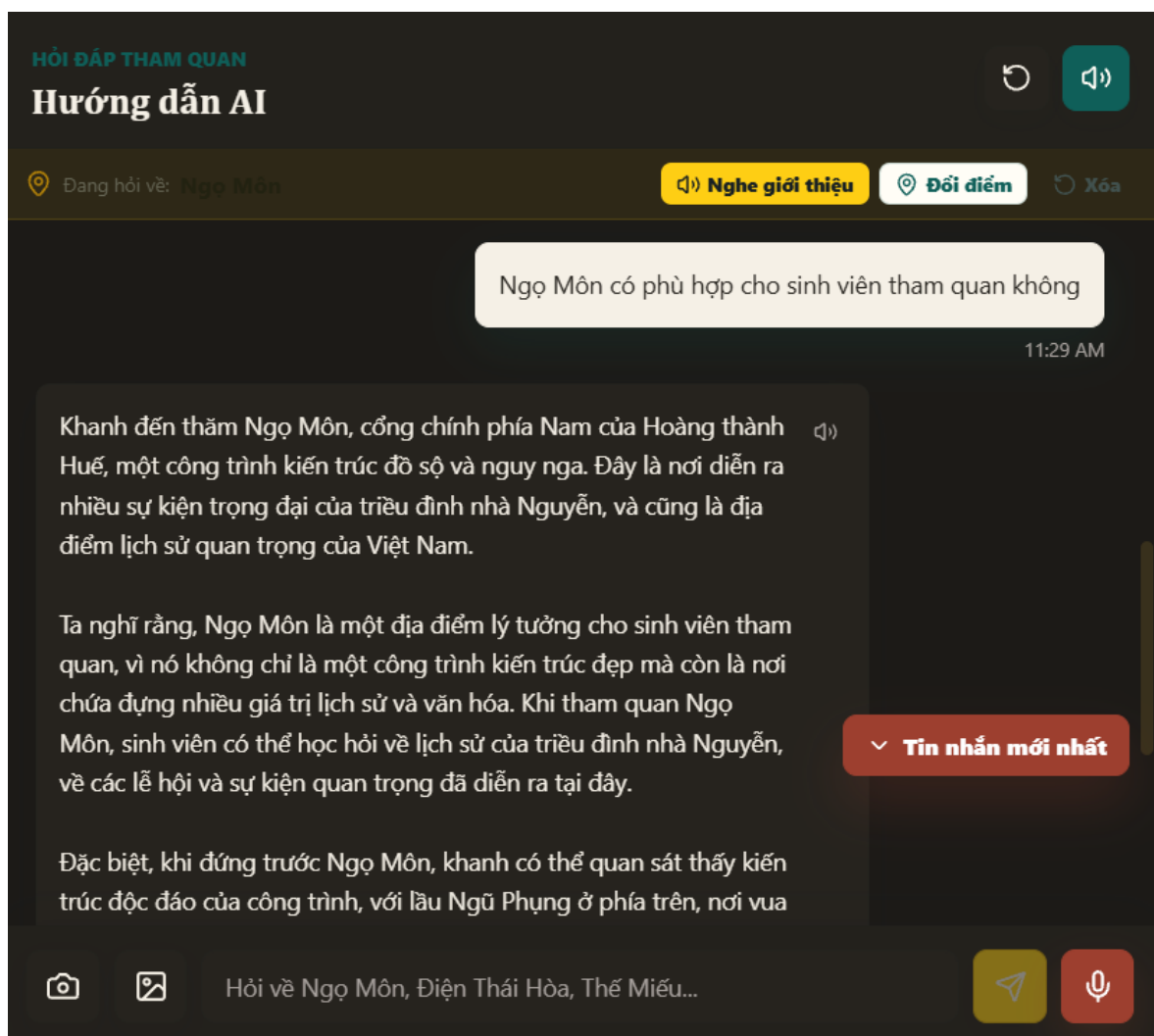
- **Thành phần Bản đồ (Leaflet):** Hiển thị polyline phân định ranh giới Đại Nội Huế. Nút "My Location" (Góc phải dưới) dùng Web Geolocation API để track vị trí user và vẽ một chấm xanh nhấp nháy.

- **Markers & Bottom Sheet:** Khi bấm vào một icon di tích (Marker) trên bản đồ, một thẻ thông tin (Bottom Sheet) sẽ trượt từ dưới lên hiển thị tên, ảnh thumbnail và khoảng cách từ chỗ đứng hiện tại đến di tích đó. Nút "Chỉ đường" (Navigate) sẽ gọi API OSRM để vẽ đường đi bộ trực tiếp trên map.

10.3. Màn hình Trợ lý Đa phương thức

[Người viết: Mai Thúc Hải Đăng - 24120276]

Đây là giao diện tương tác cốt lõi, nơi người dùng sử dụng Text, Voice và Vision để giao tiếp với AI.



Hình 10: Giao diện chat với AI

Giải thích UI/UX:

- **Bong bóng chat:** Phân biệt rõ màu sắc giữa User và AI. Câu trả lời của AI

luôn có một icon "Loa"(Speaker) bên cạnh để kích hoạt Web Speech API đọc văn bản thành tiếng.

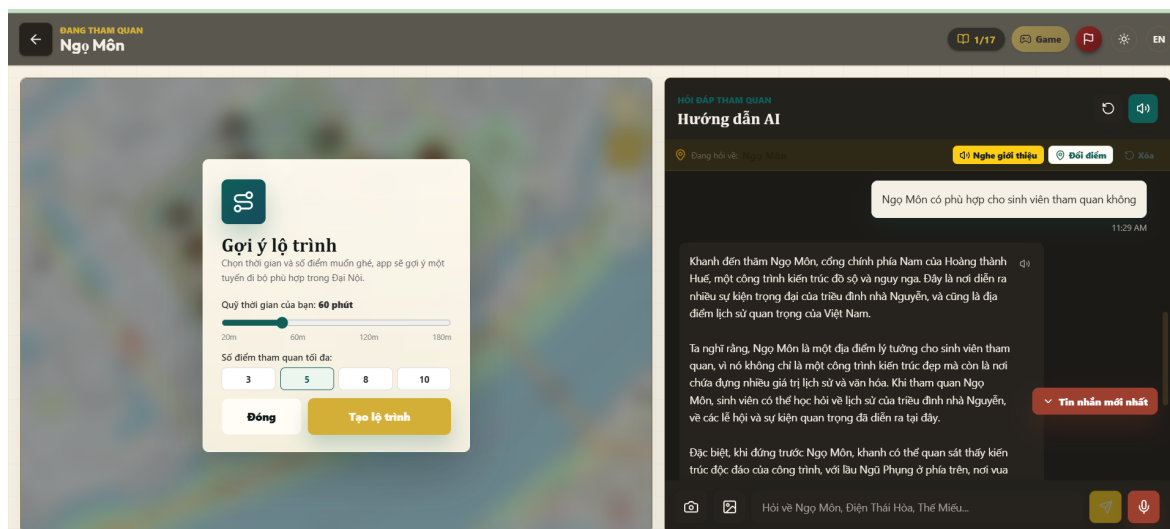
- **Khu vực nhập liệu:**

- *Nút Microphone:* Nhấn giữ để thu âm. Giao diện sẽ hiển thị hiệu ứng sóng âm (Waveform) để báo hiệu hệ thống đang nghe.
- *Nút Camera:* Mở thư viện ảnh hoặc trình chụp ảnh của điện thoại. Ảnh sau khi chọn sẽ hiện dưới dạng thumbnail nhỏ trong khung chat trước khi nhấn Gửi.

10.4. Màn hình Lập lịch trình

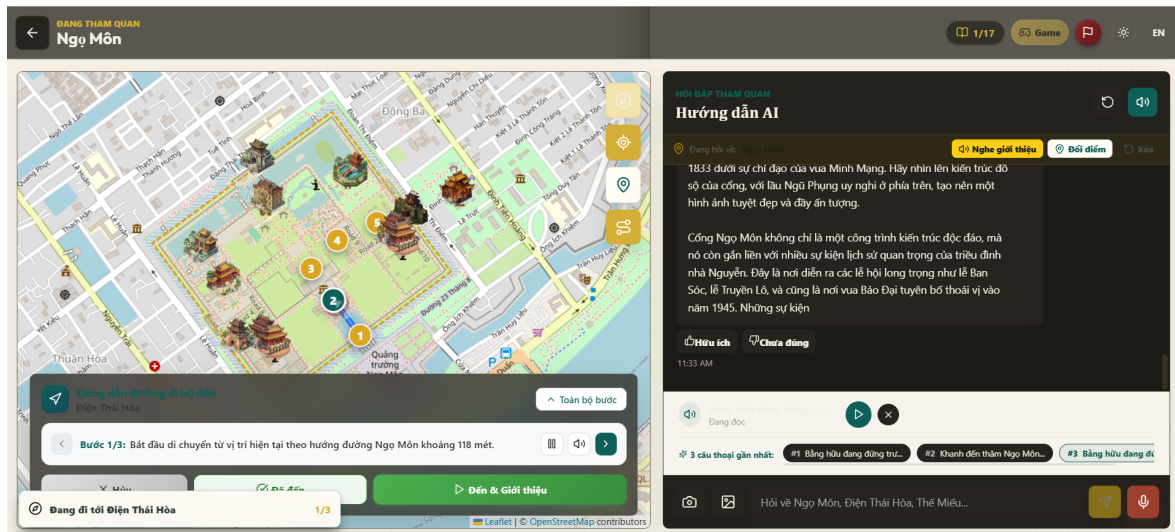
[Người viết: Mai Thúc Hải Đăng - 24120276]

Giao diện dành cho người dùng muốn hệ thống tự động tạo lộ trình tham quan dựa trên quỹ thời gian rảnh.



Hình 11: Giao diện tạo lịch trình tự động

Khi chọn xong các thông tin cần thiết, hệ thống sẽ dẫn đường đến các địa điểm đã được đánh dấu trên bản đồ



Hình 12: Giao diện tạo lịch trình tự động

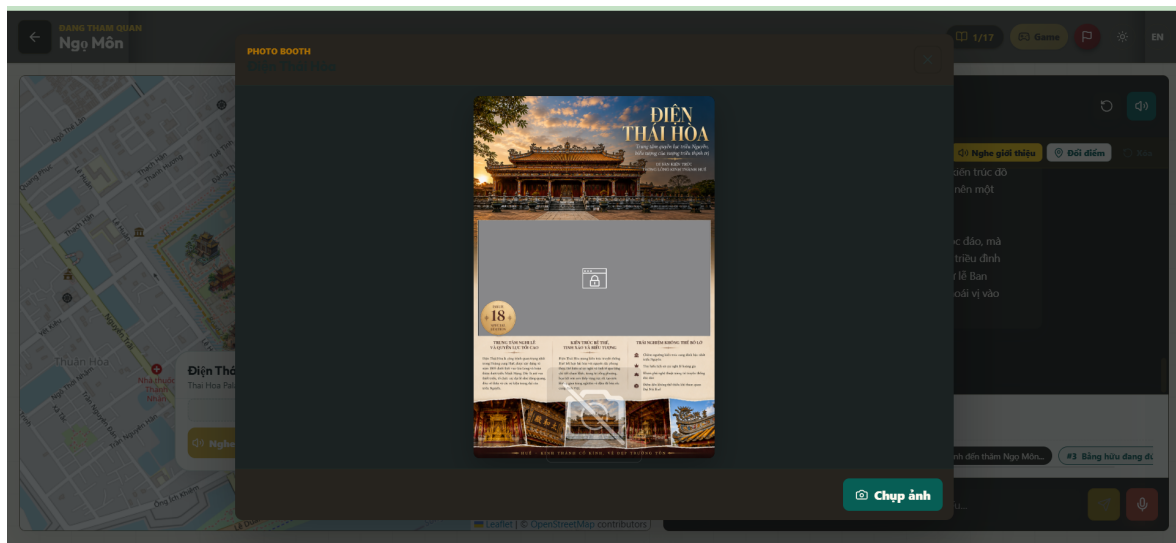
Giải thích UI/UX:

- **Input Form:** Một thanh trượt (Slider) cho phép chọn thời gian (từ 1 đến 4 tiếng) và điểm xuất phát.
- **Timeline View (Trình bày dạng trục thời gian):** Sau khi thuật toán Greedy xử lý xong ở Backend, Frontend sẽ render một trục thời gian dọc (Vertical Timeline) mô tả chi tiết: "08:00 - Điểm A (30p) → 08:35 - Di bộ 5p đến Điểm B...".

10.5. Màn hình check in

[Người viết: Mai Thúc Hải Đăng - 24120276]

Khi lựa chọn check in địa điểm đó, hệ thống sẽ tự động bật camera với khung nền là địa điểm đang chọn giống như photo booth, người dùng có thể chụp ảnh và lưu lại để làm kỉ niệm

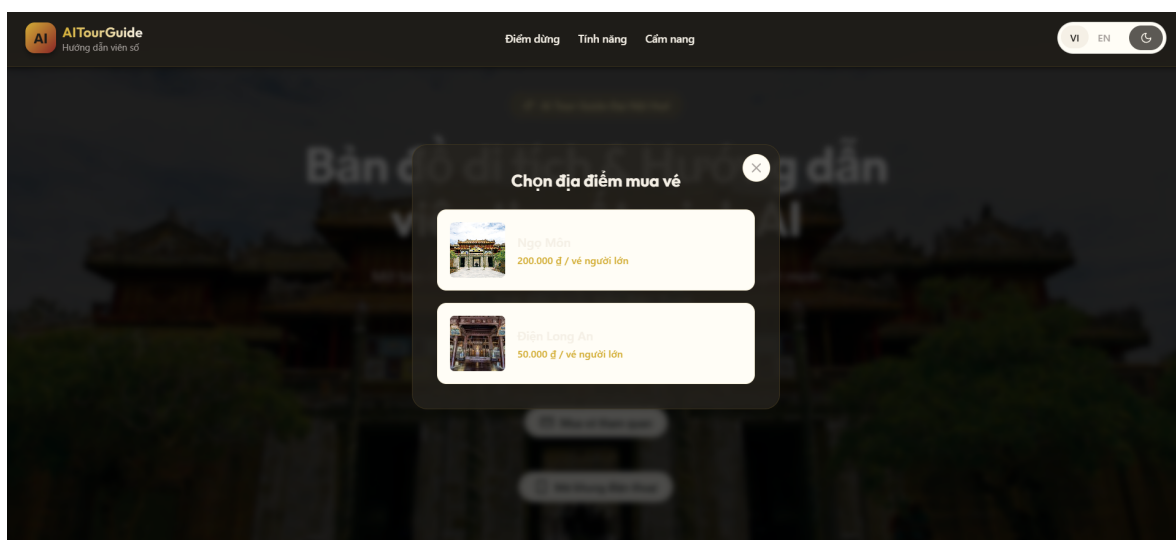


Hình 13: Giao diện check in

10.6. Màn hình mua vé tham quan

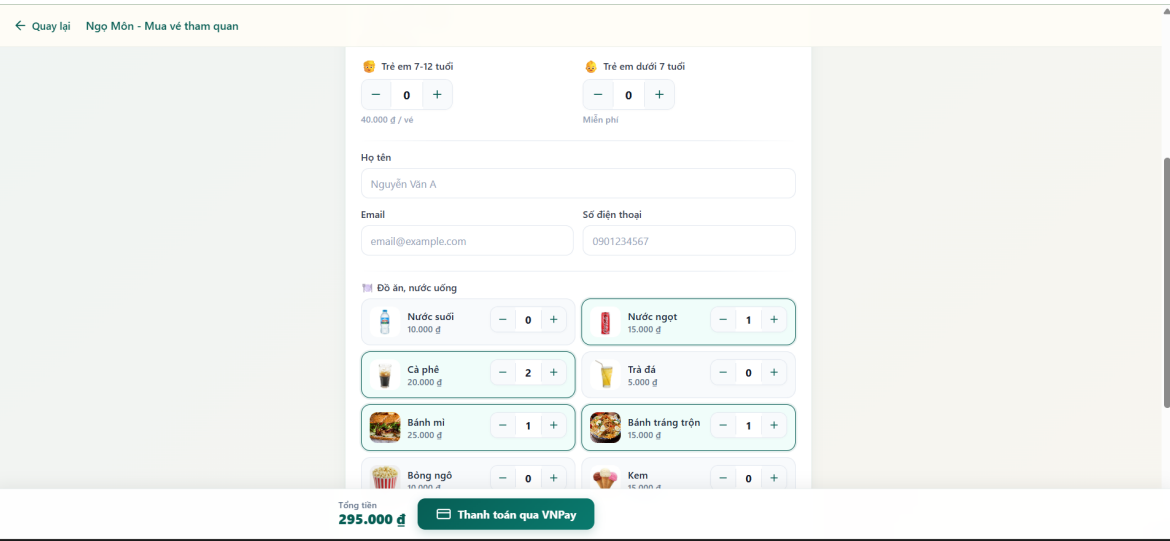
[Người viết: Mai Thúc Hải Đăng - 24120276]

Khi lựa chọn mua vé tham quan, người dùng sẽ có 2 lựa chọn mua vé gồm Ngọ Môn và Điện Long An do Đại Nội Huế chỉ bán vé cho 2 địa điểm này



Hình 14: Giao diện mua vé tham quan

Khi chọn địa điểm muốn mua vé, màn hình sẽ chuyển đến giao diện chi tiết để người dùng thao tác.



Hình 15: Giao diện mua vé tham quan

10.7. Màn hình quiz game

[Người viết: Mai Thúc Hải Đăng - 24120276]

Sau khi tham quan xong 2 địa điểm trở lên, người dùng có thể lựa chọn chơi quiz game để ôn lại những gì đã trải nghiệm được



Hình 16: Giao diện chơi game

11. Deployment

Do giới hạn về ngân sách duy trì API Key (Gemini, Groq), nhóm quyết định đóng gói và triển khai hệ thống trên môi trường cục bộ (Localhost). Việc triển khai cục bộ vẫn

đảm bảo tuân thủ kiến trúc phân tách module nhiều lớp (modular architecture), tách biệt hoàn toàn các tiến trình Frontend, Backend và Database.

11.1. System Prerequisites (Yêu cầu hệ thống)

[Người viết: Mai Thúc Hải Đăng - 24120276]

Để khởi chạy hệ thống AI Tour Guide trên môi trường local, máy chủ (hoặc máy chấm thi) cần cài đặt sẵn các môi trường sau:

- **Node.js (v18.0+)**: Môi trường runtime để build và chạy Frontend React/Vite.
- **Python (v3.10+)**: Môi trường để chạy các lệnh quản lý Backend FastAPI (ví dụ: tạo file migration).
- **Docker Desktop**: Bắt buộc phải có Docker Engine và Docker Compose (v2+) để chạy cụm container PostgreSQL (sử dụng image `pgvector/pgvector:pg16`) và FastAPI Backend đồng bộ.

11.2. Execution Steps (Quy trình khởi chạy hệ thống)

[Người viết: Trần Nguyên Tân - 24120438]

Nhóm đã thiết lập sẵn tệp cấu hình Docker Compose để tự động hóa quy trình khởi chạy, tránh lỗi xung đột môi trường. Dưới đây là luồng triển khai cục bộ từng bước:

Bước 1: Khởi chạy Database và Backend bằng Docker Hệ thống sử dụng file `docker-compose.yml` đã cấu hình sẵn thông tin cơ sở dữ liệu (`ai_tour_guide`) và biến môi trường. Tại thư mục gốc của dự án, mở Terminal và chạy lệnh:

```
docker compose -f docker/docker-compose.yml up --build -d
```

Lệnh này sẽ tự động build image cho Backend và kéo (pull) image Database về chạy ngầm (detached mode). Sau khi chạy xong, API Gateway sẽ lắng nghe tại cổng 8000. Trang tài liệu API (Swagger UI) tự động sinh ra tại `http://localhost:8000/docs`.

Bước 2: Cập nhật cấu trúc Database (Migration) Để khởi tạo các bảng dữ liệu dựa trên schema đã định nghĩa, đảm bảo môi trường ảo của Python (`.venv`) đã được cấu hình, sau đó chạy lệnh Alembic:

```
.\.venv\Scripts\alembic upgrade head
```

Bước 3: Khởi chạy Frontend (React + Vite) Mở một Terminal (Command Prompt) mới, di chuyển vào thư mục `frontend` và chạy luồng giao diện người dùng:

```
cd frontend
npm install
npm run dev
```

Trình duyệt sẽ tự động mở ứng dụng tại địa chỉ `http://localhost:5173`. Proxy của Vite đã được thiết lập sẵn để trở luồng API sang cổng 8000 của Backend, giải quyết triệt để lỗi CORS.

11.3. Future Deployment Plan (Kế hoạch triển khai đám mây trong tương lai)

[Người viết: Trần Nguyên Tân - 24120438]

Nếu dự án được cấp ngân sách và đưa vào thực tế tại Đại Nội Huế, nhóm đã vạch ra kiến trúc triển khai Cloud hoàn chỉnh như sau:

- **Frontend:** Triển khai lên **Vercel** để tận dụng CDN toàn cầu, giúp load bản đồ Leaflet và giao diện nhanh chóng với độ trễ thấp nhất.
- **Backend & AI Core:** Đóng gói thành Docker Image và chạy trên các dịch vụ máy chủ ảo như **AWS EC2** hoặc **DigitalOcean Droplet** (Cấu hình tối thiểu 2GB RAM để xử lý mượt các luồng async gọi AI).
- **Database:** Sử dụng **Supabase** hoặc **Neon Serverless Postgres** (có hỗ trợ sẵn các extension `pgvector` và `pg_trgm`) để tối ưu chi phí vận hành và không phải tự bảo trì máy chủ DB vật lý.

12. Logbook: Plan and Actual Workload

12.1. Project Management Tools (Công cụ quản lý dự án)

[Người viết: Trần Nguyên Tân - 24120438]

Để đảm bảo luồng công việc diễn ra trơn tru giữa 6 thành viên, nhóm đã thống nhất sử dụng các công cụ sau:

- **Communication:** Sử dụng **Google Meet** để họp nhóm và **Zalo** để trao đổi tin nhắn nhanh hàng ngày.
- **Project Management:** Sử dụng Google Docs để phân công các task và theo dõi tiến độ làm việc.

- **Source Code Management:** Sử dụng **GitHub**. Nhóm quy định rõ ràng việc tạo nhánh cho từng tính năng và phải tạo Pull Request (PR) để review code trước khi merge vào nhánh `main`.
- **Document Management:** Sử dụng **Google Drive** để lưu trữ API Keys, JSON data thô; và dùng **Prism** để cùng nhau biên soạn báo cáo LaTeX này.

12.2. Plan Workload

[Người viết: Trần Nguyên Tân - 24120438]

Theo kế hoạch ban đầu (được lập vào tuần 2 của đề án), nhóm dự kiến hoàn thành dự án trong 8 tuần với phân bổ thời gian như sau:

- **Tuần 1-2 (Requirement & Design):** Phân tích bài toán, thiết kế C4 Model, thiết kế schema Database. (Dự kiến: Dễ, xong sớm).
- **Tuần 3-4 (Backend & Database):** Setup FastAPI, chạy Docker cho PostgreSQL, viết các endpoint CRUD cơ bản.
- **Tuần 5 (AI Integration):** Tích hợp Gemini, Groq và viết hàm Vector Search. (Dự kiến: Cần 1 tuần).
- **Tuần 6 (Frontend & Map):** Dựng UI bằng React, tích hợp Leaflet, ghép nối API.
- **Tuần 7 (Testing & Bug Fixing):** Test thực tế và sửa lỗi.
- **Tuần 8 (Report & Demo):** Viết báo cáo Overleaf, quay video demo.

12.3. Actual Workload

[Người viết: Trần Nguyên Tân - 24120438]

Thực tế triển khai có sự chênh lệch đáng kể so với kế hoạch ban đầu:

- **Giai đoạn Design:** Khớp với kế hoạch.
- **Giai đoạn AI Integration (Bị trễ 1.5 tuần):** Thực tế việc gọi API của Gemini thường xuyên bị lỗi Timeout, và STT nhận diện sai quá nhiều. Nhóm phải tốn thêm rất nhiều thời gian để viết thuật toán `UnifiedOrchestrator` (bất đồng bộ) và Fuzzy Search để sửa lỗi chính tả.

- **Giai đoạn Frontend (Khớp tiến độ):** Giao diện được dựng khá nhanh nhờ dùng TailwindCSS, nhưng phần ghép polyline lên bản đồ Leaflet bị lệch tọa độ mất thêm 3 ngày để fix.
- **Giai đoạn Testing:** Do AI integration trễ, nhóm phải đẩy nhanh quá trình test song song với việc viết báo cáo, dẫn đến tổng số giờ làm việc thực tế cao hơn dự kiến ban đầu (khoảng 400 giờ tổng cộng cho cả nhóm).

12.4. Workload Distribution (Bảng phân chia công việc)

[Người viết: Trần Nguyên Tân - 24120438]

Tổng khối lượng công việc (100%) được phân chia cho 6 thành viên dựa trên thể mạnh của từng người.

MSSV	Họ và Tên	Trách nhiệm chính (Roles)	Tỷ lệ (%)	Số giờ làm
24120276	Mai Thúc Hải Đăng	Backend Lead: Code FastAPI, thiết kế kiến trúc xử lý Async, quản lý Orchestrator.	17%	68h
24120300	Đỗ Ngọc Hải	Database & RAG: Thiết kế Schema, setup pgvector, thuật toán Semantic Search.	17%	68h
24120349	Bùi Tấn Kiên	Frontend Lead: Dựng UI React/Vite, xử lý State Management và tích hợp Map Leaflet.	17%	68h
24120438	Trần Nguyên Tân	AI Integration: Tích hợp API Gemini/Groq, viết prompt, xử lý STT/Vision logic.	17%	68h
24120427	Võ Trúc Quỳnh	UI/UX & Testing: Thiết kế Wireframe, xử lý luồng âm thanh TTS, Unit Test, QA.	16%	64h
24120486	Trần Nhật Trường	Routing & Document: Code thuật toán Tour Planning, quản lý OSRM, tổng hợp Báo cáo.	16%	64h
Tổng cộng (Total)			100%	400h

Bảng 7: Bảng phân chia khối lượng công việc thực tế (Actual Workload)

13. AI usage and declaration

13.1. Tích hợp AI trong hệ thống

[Người viết: Trần Nguyên Tân - 24120438]

13.1.1. Chiến lược Tích hợp Trí tuệ Nhân tạo Đa luồng

[Người viết: Trần Nguyên Tân - 24120438]

Điểm đột phá của kiến trúc hệ thống nằm ở việc không sử dụng một mô hình chatbot nguyên khối (monolithic) duy nhất. Thay vào đó, nhóm áp dụng chiến lược phân tách vi dịch vụ (microservices), sử dụng nhiều mô hình AI chuyên biệt cho từng tác vụ cụ thể. Điều này giúp hệ thống duy trì nhận thức ngữ cảnh không gian và hỗ trợ tương tác đa phương thức (văn bản, giọng nói, hình ảnh) một cách mượt mà và tối ưu nhất.

13.1.2. Ứng dụng Google Gemini cho Xử lý Thị giác, Ngôn ngữ và Nhúng (Embedding)

[Người viết: Trần Nguyên Tân - 24120438]

Hệ thống sử dụng hệ sinh thái Google Gemini làm lõi xử lý AI chính nhờ khả năng giải quyết các tác vụ đa phương thức vượt trội:

- **Xử lý ảnh (Vision):** Sử dụng Gemini Vision để phân tích hình ảnh du khách tải lên hoặc chụp từ camera. Nhóm thiết lập các prompt đặc tả nghiêm ngặt để mô hình nhận diện các đặc trưng kiến trúc và hiện vật trong Đại Nội. Kết quả nhận diện thô không được trả trực tiếp cho người dùng, mà được sử dụng làm từ khóa để truy vấn đối chiếu với cơ sở dữ liệu, đảm bảo thông tin lịch sử xuất ra là chính xác tuyệt đối.
- **Tạo câu trả lời (Text Generation):** Gemini Text đóng vai trò điều phối trung tâm. Mô hình tiếp nhận ý định của người dùng, kết hợp với dữ kiện lịch sử truy xuất từ Database (qua luồng RAG) và lịch sử hội thoại. Từ đó, tổng hợp thành một phản hồi tự nhiên, chuyên nghiệp theo đúng văn phong của một hướng dẫn viên thực thụ.
- **Tạo không gian Vector (Embeddings):** Nhằm phục vụ cơ chế RAG, mô hình Gemini Embedding được dùng để chuyển đổi các văn bản lịch sử và bộ câu hỏi thường gặp (FAQs) thành các vector đa chiều, sau đó lưu trữ vào PostgreSQL

thông qua `pgvector`. Khi nhận được truy vấn, hệ thống nhúng câu hỏi thành vector và tính toán khoảng cách Cosine (Cosine Similarity) để tìm kiếm thông tin liên quan nhất, qua đó triệt tiêu hiện tượng “ảo giác” (hallucination) thường gặp ở các LLM.

13.1.3. Tích hợp Whisper cho Chuyển đổi Giọng nói (Speech-to-Text)

[Người viết: Trần Nguyên Tân - 24120438]

Để tối ưu trải nghiệm rảnh tay (hands-free) cho du khách khi di chuyển, hệ thống tích hợp mô hình Whisper thông qua API của Groq. Khi người dùng thu âm, luồng Frontend sẽ truyền tải tệp âm thanh về Backend để xử lý qua nền tảng này. Mô hình Whisper có khả năng nhận diện tiếng Việt và xuất ra văn bản với độ chính xác cao. Việc tận dụng bộ vi xử lý LPU (Language Processing Unit) chuyên dụng của Groq giúp quá trình bóc băng (transcribe) diễn ra gần như tức thời, giảm thiểu tối đa độ trễ cho toàn bộ chu trình giao tiếp.

13.1.4. Cơ chế Dự phòng (Fallback Mechanism) với Llama-3

[Người viết: Trần Nhật Trường - 24120486]

Nhằm đảm bảo tính sẵn sàng cao (High Availability), nhóm triển khai thêm mô hình Llama-3 (thông qua nền tảng Groq) làm hệ thống dự phòng. Lớp `FallbackLLMProvider` được thiết kế để liên tục giám sát trạng thái của Google Gemini. Trong trường hợp API chính bị quá tải, giới hạn tỷ lệ (Rate Limit) hoặc phản hồi lỗi, hệ thống sẽ tự động rẽ lối (route) truy vấn sang Llama-3. Cơ chế này đảm bảo trợ lý ảo luôn duy trì khả năng phản hồi liên tục mà không làm gián đoạn trải nghiệm của du khách.

13.1.5. Kỹ nghệ Prompt (Prompt Engineering) và Quản lý Ngữ cảnh (Context)

[Người viết: Trần Nhật Trường - 24120486]

Hiệu suất và độ tin cậy của trợ lý ảo phụ thuộc rất lớn vào kỹ thuật tinh chỉnh chỉ thị (Prompt Engineering). Nhóm thiết lập các bộ quy tắc hệ thống (System Prompts) khắt khe nhằm giới hạn phạm vi sinh văn bản của AI, ngăn chặn việc mô hình đi lạc đề. Các kỹ thuật như Few-Shot Prompting được áp dụng bằng cách cung cấp các ví dụ mẫu để định chuẩn giọng điệu và cấu trúc phản hồi.

Bên cạnh đó, việc quản lý mạch hội thoại (Context Management) là một thách thức được xử lý kỹ lưỡng. Backend thực hiện lưu trữ toàn bộ lịch sử trò chuyện cục bộ, sau đó trích xuất thông tin lịch sử liên quan và tọa độ địa lý hiện tại của người dùng để tiêm (inject) vào câu lệnh cuối cùng. Cơ chế này giúp AI duy trì nhận thức toàn diện về cả không gian lẫn mạch giao tiếp.

13.2. DECLARATION

[Người viết: Trần Nhật Trường - 24120486]

Trong quá trình làm đồ án, nhóm chúng em có sử dụng công cụ AI để hỗ trợ code với các prompt sau đây:

1. **Câu prompt:** Làm ơn tái cấu trúc lại toàn bộ dự án, chuyển backend sang dạng module
hóa với FastAPI, thêm frontend Vite + React có bản đồ và voice chat,
cập nhật dữ liệu 17 địa điểm, tăng token LLM lên 2048, thêm Docker
và Alembic, xóa code cũ
2. **Câu prompt:** Cải thiện pipeline LLM lên: tăng temperature, thêm retry cho Gemini khi
lỗi, bỏ giới hạn từ cho Groq, thêm timeout, viết lại prompt kể chuyện
hấp dẫn hơn 300-500 từ, thêm cache cho mô tả artifact, hỗ trợ hỏi nhiều
địa điểm cùng lúc, viết lại mô tả 17 địa điểm chi tiết lịch sử văn hóa
3. **Câu prompt:** Thêm thư viện ảnh cho điểm đến với lightbox phóng to, vuốt
qua ảnh,
cải tiến tuyến đường trên bản đồ cho đẹp hơn, hiển thị giờ mở cửa
4. **Câu prompt:** Tích hợp thanh toán VNPay để mua vé online, thêm đánh giá
sao 3 tiêu chí
(dịch vụ, cảnh quan, giá vé) kèm nhận xét, thêm đặt đồ ăn thức uống,
xây API backend thanh toán và đánh giá, tạo giao diện checkout cho
Ngọ Môn và Điện Long An
5. **Câu prompt:** Làm lại giao diện cho đẹp hơn: thêm bottom bar cố định có tổng
tiền

và nút thanh toán, đồ ăn sang grid 2 cột, thêm animation mượt, progress bar gradient, avatar cho đánh giá, glassmorphism header, sửa mã VNPay trong .env

6. **Câu prompt:** Tôi đang đập đi xây lại một ứng dụng điện thoại về du lịch. Hãy đóng vai trò là Product Manager, Solution Architect và AI Engineer để lập một kế hoạch xây dựng ứng dụng bằng Markdown.

Bối cảnh sản phẩm

Tôi muốn xây dựng một ứng dụng “AI Virtual Tour Guide” — tức là một hướng dẫn viên du lịch ảo sử dụng AI. Ban đầu app chỉ cần hỗ trợ một địa điểm du lịch cụ thể, ví dụ Dinh Độc Lập hoặc Kinh thành Huế. Bên trong địa điểm chính sẽ có nhiều điểm tham quan con như cổng vào, sân chính, khu trưng bày, phòng lịch sử, công trình kiến trúc, điểm check-in, khu ăn uống, nhà vệ sinh, lối ra, v.v.

Người dùng sẽ bắt đầu ở một điểm trong địa điểm đó. Sau đó họ có thể chọn đi đến một điểm khác. Hệ thống sẽ hướng dẫn đường đi, gợi ý lộ trình tham quan, giải thích tại sao nên đi theo lộ trình đó, và khi người dùng đến từng điểm thì AI sẽ giới thiệu địa điểm bằng giọng nói như một hướng dẫn viên thật.

Người dùng có thể đặt câu hỏi bằng giọng nói hoặc văn bản, ví dụ:

“Chỗ này có ý nghĩa gì?”

“Tại sao công trình này nổi tiếng?”

“Điểm tiếp theo nên đi đâu?”

“Ở gần đây có quán ăn nào không?”

“Nếu tôi chỉ còn 30 phút thì nên đi những điểm nào?”

Ứng dụng cần mô phỏng trải nghiệm một chuyến đi du lịch thật có hướng dẫn viên, không chỉ là app bản đồ hoặc chatbot.

Yêu cầu trải nghiệm người dùng

Hãy thiết kế app theo hướng user-oriented. Tập trung vào việc người dùng đi du

lich được trải nghiệm trước khi tham quan chính thức

App cần có các trải nghiệm:

Gợi ý lộ trình tham quan theo thời gian, sở thích và sức khỏe của người dùng.

Hướng dẫn đường đi giữa các điểm tham quan con.

Giới thiệu từng điểm bằng giọng nói.

Cho phép hỏi đáp tự nhiên với AI.

Gợi ý quán ăn, chỗ nghỉ

Có chế độ xem bản đồ, xem ảnh, xem 360 độ hoặc 3D.

Có thể dùng hình ảnh 3D từ API nếu khả thi.

Nếu nâng cao, người dùng có thể tương tác với ảnh 3D/360 độ, xoay góc nhìn, bấm vào hotspot, hoặc chọn điểm tiếp theo trực tiếp trên không gian ảo.

Yêu cầu kỹ thuật

Hãy đề xuất kiến trúc tổng thể cho mobile app, backend, database, AI service, bản đồ, định vị, voice, RAG và hệ thống dữ liệu địa điểm.

Hãy cân nhắc các công nghệ/API có thể dùng như:

Google Maps Platform, Places API, Routes API, Street View, 3D Maps hoặc Photorealistic 3D Tiles.

Mapbox/Cesium nếu phù hợp.

LLM cho hội thoại.

RAG để trả lời dựa trên dữ liệu địa điểm đáng tin cậy.

STT/TTS để người dùng nói chuyện với AI.

Vector database cho tri thức du lịch.

Tour graph để mô hình hóa các điểm tham quan và đường đi giữa chúng.

Yêu cầu về plan

Hãy lập plan bằng Markdown, có cấu trúc rõ ràng gồm:

1. Tầm nhìn sản phẩm.
2. Người dùng mục tiêu.
3. Vấn đề app giải quyết.
4. Trải nghiệm người dùng lý tưởng.

5. Danh sách tính năng bắt buộc cho MVP.
6. Danh sách tính năng nâng cao.
7. Mô hình dữ liệu địa điểm và điểm tham quan con.
8. Kiến trúc hệ thống.
9. Luồng xử lý chính của app.
10. Công nghệ/API đề xuất và lý do chọn.
11. Thuật toán hoặc logic cần dùng: route planning, tour graph, recommendation, RAG, ranking địa điểm.
12. Thiết kế AI hướng dẫn viên: persona, giọng nói, cách trả lời, cách xử lý câu hỏi ngoài phạm vi.
13. Quy tắc xây dựng sản phẩm: user-oriented, accessibility, privacy, offline-first nếu cần, reliability, cost-aware, safety, explainability.
14. Roadmap phát triển theo từng giai đoạn.
15. Rủi ro kỹ thuật/pháp lý và cách giảm thiểu.
16. Tiêu chí đánh giá thành công của MVP.
17. Gợi ý demo đầu tiên với một địa điểm cụ thể như Kinh thành Huế hoặc Dinh Độc Lập.

Hãy suy nghĩ như đang xây một sản phẩm thật, không chỉ liệt kê tính năng. Ưu tiên giải pháp có thể triển khai được cho MVP, nhưng vẫn đề xuất hướng nâng cấp để tạo hiệu ứng “wow”.

7. **Câu prompt:** Tôi muốn bạn đọc và phân tích toàn bộ codebase hiện tại của project trước, bao gồm cả việc kiểm tra các Agent Skills, agent instructions, repository guidelines, coding conventions, architecture documents, README, docs, workflows, playbooks hoặc bất kỳ tài liệu hướng dẫn nào liên quan đến cách làm việc trong project.

Sau khi hiểu codebase, hãy tạo một file kế hoạch thiết kế lại UI mobile bằng Markdown.

Mục tiêu chính:

Thiết kế một UI phù hợp cho mobile.

Ưu tiên trải nghiệm người dùng thực tế, user-oriented và user-centric.

Giao diện cần có cá tính riêng, thân thiện, rõ ràng, dễ sử dụng.

Tránh tạo giao diện kiểu AI công nghiệp, dashboard kỹ thuật, admin panel

hoặc UI quá máy móc nếu không phù hợp với sản phẩm.

Không áp đặt ý tưởng thiết kế khi chưa hiểu sản phẩm.

Hãy để đặc thù sản phẩm được suy ra từ codebase, README, docs và các màn hình/chức năng hiện có.

Cố gắng không can thiệp backend nếu không thật sự cần thiết.

Không tự ý sửa code implementation trong bước này.

Chỉ tạo file plan bằng Markdown.

Yêu cầu bắt buộc trước khi viết plan:

1. Đọc cấu trúc codebase.
2. Đọc và tuân thủ các Agent Skills, agent instructions, repository guidelines, coding conventions, architecture documents, README, docs hoặc bất kỳ tài liệu hướng dẫn nào liên quan.
3. Xác định project đang làm sản phẩm gì.
4. Xác định người dùng chính của sản phẩm là ai dựa trên codebase và tài liệu hiện có.
5. Xác định frontend đang dùng công nghệ gì.
6. Xác định các màn hình, component, routing, layout, state management và API calls hiện tại.
7. Phân biệt rõ phần nào thuộc UI/frontend, phần nào thuộc backend/API.
8. Xác định các ràng buộc kỹ thuật hiện tại của frontend.
9. Nếu có điểm chưa rõ, hãy ghi vào phần Assumptions hoặc Open Questions thay vì tự ý suy diễn quá mức.
10. Trước khi lập kế hoạch, hãy tóm tắt ngắn gọn những gì đã học được từ codebase và các tài liệu/Agent Skills liên quan.

File cần tạo:

Tạo file Markdown tên: 'MOBILE_UI_REDESIGN_PLAN.md'

Đặt ở thư mục gốc của project.

Nếu repo đã có thư mục 'docs/', hãy đặt file trong 'docs/'.

Nội dung file Markdown cần có cấu trúc sau:

```
# Mobile UI Redesign Plan
```


1. Codebase Understanding

Tóm tắt những gì đã hiểu sau khi đọc codebase:

Project này là gì.

Người dùng chính có thể là ai.

Frontend đang dùng công nghệ gì.

Backend/API đang đóng vai trò gì.

Các thư mục/file quan trọng liên quan đến UI.

Các màn hình/chức năng chính đang có.

Các component/layout chính đang có.

Routing hiện tại.

State management hiện tại nếu có.

Các API/backend endpoint đang được UI gọi đến.

Những phần backend không nên động vào.

Các Agent Skills, conventions hoặc project guidelines có ảnh hưởng đến việc triển khai UI.

2. Current UI/UX Problems

Phân tích các vấn đề hiện tại của UI theo góc nhìn mobile:

Layout

Navigation

Information architecture

Mật độ thông tin

Khả năng thao tác bằng ngón tay

Độ rõ ràng của nội dung

Tính nhất quán giữa các màn hình

Visual hierarchy

Responsive behavior

Loading/error/empty states

Accessibility cơ bản

Cảm giác thương hiệu/sản phẩm

Các điểm có thể gây nhầm lẫn cho người dùng

Nếu chưa đủ thông tin để kết luận, hãy ghi rõ là cần xác minh thêm.

3. Product and User Direction

Dựa trên codebase, hãy suy luận định hướng sản phẩm:

Sản phẩm phục vụ mục đích gì.

Người dùng mở app để làm gì.

Các nhu cầu chính của người dùng.

Các luồng sử dụng quan trọng.

Cảm xúc mà giao diện nên tạo ra.

Giao diện nên tránh cảm giác gì.

Những yếu tố nào nên được ưu tiên trên mobile.

Những yếu tố nào nên được giảm bớt hoặc ẩn đi trên mobile.

Không áp đặt domain từ bên ngoài. Hãy dựa vào codebase và tài liệu hiện có.

4. Mobile-First UX Principles

Đưa ra các nguyên tắc thiết kế UI mobile cho project này:

Mobile-first, không chỉ là responsive lại từ desktop.

Mỗi màn hình nên có một hành động chính rõ ràng.

Nội dung phải ngắn, dễ quét mắt, dễ hiểu.

Ưu tiên thao tác nhanh.

Thiết kế cho thao tác bằng một tay nếu phù hợp.

Button, input và vùng chạm phải đủ lớn.

Navigation phải rõ ràng, dễ quay lại, dễ hiểu.

Ưu tiên bottom navigation hoặc bottom actions nếu phù hợp.

Ưu tiên bottom sheet thay vì modal desktop nếu phù hợp.

Không dùng layout nhiều cột kiểu desktop trên mobile.

Không nhồi quá nhiều thông tin trên một màn hình.

Dùng progressive disclosure để giấu bớt thông tin phụ.

Loading, empty, error và offline states phải thân thiện.

Microcopy phải tự nhiên, hướng người dùng, không quá kỹ thuật.

Tránh hiển thị thông tin kỹ thuật nội bộ cho người dùng phổ thông nếu không cần thiết.

5. Information Architecture

Đề xuất lại cấu trúc thông tin cho mobile:

Các tab chính nên có nếu app phù hợp với bottom navigation.

Các màn hình chính.

Các màn hình phụ.

Các màn hình detail.

Các modal/bottom sheet cần có.

Luồng onboarding nếu cần.

Luồng sử dụng hằng ngày.

Luồng xử lý lỗi.

Luồng khi chưa có dữ liệu.

Luồng khi mất mạng hoặc API lỗi.

Với mỗi màn hình/nhóm màn hình, mô tả:

Mục đích.

Người dùng cần làm gì ở đây.

Nội dung chính.

Hành động chính.

Hành động phụ.

Component nên dùng.

Dữ liệu/API cần dùng.

Có cần backend thay đổi không.

6. Screen Priority

Xác định mức độ ưu tiên của các màn hình trên mobile:

Primary Screens

Các màn hình cốt lõi người dùng sẽ dùng thường xuyên.

Với mỗi màn hình, ghi rõ:

Vì sao quan trọng.

Nên nằm ở đâu trong navigation.

Hành động chính là gì.

Dữ liệu cần hiển thị.

Backend impact: None / Low / Medium / High.

Secondary Screens

Các màn hình cần có nhưng không nên đặt ngang hàng với màn hình chính.

Detail Screens

Các màn hình chỉ nên mở khi người dùng chọn một item cụ thể.

Utility Screens

Các màn hình cài đặt, profile, lịch sử, trợ giúp hoặc các chức năng phụ.

Features to De-emphasize on Mobile

Các tính năng hiện có nhưng không nên đưa lên quá nổi bật trên mobile vì có thể gây rối, quá tải hoặc không phù hợp với người dùng phổ thông.

7. Screen-by-Screen Redesign

Viết chi tiết từng màn hình theo format:

Screen Name

User goal:

Current problem:

Main content:

Primary action:

Secondary actions:

Suggested layout:

Navigation behavior:

UI components:

Loading state:

Empty state:

Error state:

Accessibility notes:

Related existing files:

Related API/data:

Backend impact: None / Low / Medium / High

Implementation notes:

8. Mobile Interaction Patterns

Đề xuất các pattern tương tác phù hợp:

Bottom navigation

Floating action button nếu phù hợp

Sticky bottom CTA

Bottom sheet

Pull to refresh

Swipe actions nếu phù hợp

Search and filter

Step-by-step flow

Expand/collapse content

Toast/snackbar

Confirmation dialog

Permission request flow

Offline/retry flow

Với mỗi pattern, hãy ghi rõ:

Dùng ở đâu.

Vì sao phù hợp.

Tránh lạm dụng như thế nào.

9. Visual Design Direction

Đề xuất phong cách giao diện dựa trên sản phẩm hiện có:

Mood/tone.

Typography direction.

Spacing.

Layout rhythm.

Color direction.

Icon style.

- Card style.
- Button style.
- Input style.
- Empty state illustration/icon direction.
- Motion/animation nếu phù hợp.
- Cách tạo cảm giác khác biệt nhưng vẫn dễ dùng.

Không cần chọn màu cụ thể nếu codebase chưa có design system, nhưng hãy đề xuất hướng thiết kế rõ ràng.

Nếu codebase đã có theme/design system, hãy ưu tiên tận dụng thay vì tạo mới hoàn toàn.

10. Content and Microcopy Direction

Đề xuất cách viết nội dung trong UI:

- Ngắn gọn.
- Dễ hiểu.
- Gần với ngôn ngữ người dùng.
- Không quá kỹ thuật.
- Không dùng từ mơ hồ cho button.
- Button phải thể hiện hành động cụ thể.
- Error message phải giải thích được vấn đề và gợi ý bước tiếp theo.
- Empty state phải cho người dùng biết họ có thể làm gì tiếp.
- Loading state không nên gây cảm giác app bị treo.

Đưa ra ví dụ chung:

- Bad microcopy:
- Better microcopy:

Chỉ dùng ví dụ phù hợp với domain đã suy ra từ codebase.

11. Component System Plan

Đề xuất các component frontend nên có hoặc nên chuẩn hóa:

- App layout

Screen container
Header
Bottom navigation
Tab bar
Search/input
Primary button
Secondary button
Icon button
Card
List item
Detail section
Empty state
Loading state
Error state
Offline state
Modal
Bottom sheet
Toast/snackbar
Form field
Permission prompt
Skeleton loader

Với mỗi component, mô tả:

Mục đích.

Nơi sử dụng.

Props/data cần thiết ở mức khái niệm.

File hiện tại liên quan nếu có.

Có nên tạo mới, refactor hay tái sử dụng component cũ.

12. Responsive and Device Considerations

Đưa ra các tiêu chí mobile cần kiểm tra:

Small phones.

Large phones.

Tablet nếu project cần.

Safe area.

Keyboard opening behavior.

- Scroll behavior.
- Touch target size.
- Long text wrapping.
- Orientation nếu cần.
- Performance khi list dài.
- Image/media loading nếu có.
- Map/camera/microphone/location permission nếu project có dùng.
- Dark mode nếu project có hoặc nên hỗ trợ.

13. Accessibility Considerations

Đề xuất các điểm accessibility cơ bản:

- Màu có đủ tương phản.
- Text không quá nhỏ.
- Button đủ lớn.
- Icon quan trọng phải có label hoặc accessible text.
- Form field có label rõ ràng.
- Error message dễ hiểu.
- Focus order hợp lý.
- Screen reader cơ bản nếu framework hỗ trợ.
- Không chỉ dùng màu để truyền đạt trạng thái.

14. Backend-Safe Implementation Strategy

Lập kế hoạch triển khai theo hướng ít đụng backend nhất:

- Những gì chỉ cần sửa frontend.
- Những gì có thể tận dụng API hiện có.
- Những gì cần adapter/mapper ở frontend.
- Những gì có thể xử lý bằng client state.
- Những gì cần mock data tạm thời nếu API chưa có.
- Những thay đổi backend chỉ đề xuất nếu thật sự cần.
- Không phá vỡ API contract hiện tại.
- Không đổi response shape nếu không có lý do mạnh.
- Không tự ý xóa hoặc sửa endpoint backend.

Nếu có backend impact, hãy phân loại:

None: chỉ sửa UI/frontend.

Low: chỉ cần mapping hoặc sử dụng API hiện có.

Medium: cần thêm optional field hoặc endpoint phụ.

High: cần thay đổi API contract hoặc logic backend.

15. Implementation Phases

Chia thành các giai đoạn triển khai:

Phase 1: Codebase and UI Audit

Mục tiêu.

File/thư mục cần đọc.

Việc cần làm.

Output mong muốn.

Rủi ro.

Tiêu chí hoàn thành.

Phase 2: Mobile Information Architecture

Mục tiêu.

Việc cần làm.

Output mong muốn.

Rủi ro.

Tiêu chí hoàn thành.

Phase 3: Navigation and App Shell

Mục tiêu.

File/thư mục có khả năng chỉnh sửa.

Việc cần làm.

Rủi ro.

Tiêu chí hoàn thành.

Phase 4: Core Screen Redesign

Mục tiêu.

File/thư mục có khả năng chỉnh sửa.

Việc cần làm.

Rủi ro.

Tiêu chí hoàn thành.

Phase 5: Component System Cleanup

Mục tiêu.

File/thư mục có khả năng chỉnh sửa.

Việc cần làm.

Rủi ro.

Tiêu chí hoàn thành.

Phase 6: States, Microcopy and Polish

Mục tiêu.

Loading states.

Empty states.

Error states.

Offline states.

Permission states nếu có.

Microcopy.

Motion/animation nếu phù hợp.

Rủi ro.

Tiêu chí hoàn thành.

Phase 7: Testing and QA

Mục tiêu.

Mobile viewport testing.

Interaction testing.

Regression testing.

API contract checking.

Accessibility smoke test.

Performance check.

Rủi ro.

Tiêu chí hoàn thành.

16. Risks and Trade-offs

Liệt kê các rủi ro có thể gặp:

UI mới không khớp với API hiện tại.

- Component cũ khó tái sử dụng.
- Routing hiện tại không phù hợp mobile.
- State management phức tạp.
- Thiếu design system.
- Thiếu trạng thái loading/error/empty.
- Backend coupling quá chặt.
- Responsive layout hiện tại phụ thuộc desktop.
- Có thể cần refactor nhiều frontend hơn dự kiến.

Với mỗi rủi ro, đề xuất cách giảm thiểu.

17. Acceptance Criteria

Đưa ra checklist để biết plan đã thành công:

- UI hoạt động tốt trên mobile.
- Navigation rõ ràng.
- Người dùng hiểu ngay màn hình hiện tại dùng để làm gì.
- Mỗi màn hình có hành động chính rõ.
- Không có màn hình quá tải thông tin.
- Nội dung dễ đọc trên màn hình nhỏ.
- Button/input dễ chạm.
- Có loading state.
- Có empty state.
- Có error state.
- Có offline/retry state nếu phù hợp.
- Có permission state nếu project dùng quyền hệ thống.
- API contract không bị phá.
- Backend không bị thay đổi nếu không cần.
- Component được tổ chức nhất quán hơn.
- Giao diện có cá tính riêng, thân thiện, không giống dashboard kỹ thuật nếu sản phẩm không yêu cầu.
- Plan đủ chi tiết để một developer khác có thể triển khai mà không cần hỏi lại quá nhiều.

18. Assumptions and Open Questions

Liệt kê:

Các giả định được đưa ra trong quá trình phân tích.

Những điều chưa rõ sau khi đọc codebase.

Những câu hỏi cần người maintain project xác nhận.

Những điểm không nên tự ý quyết định.

Những phần cần kiểm tra thêm trước khi implementation.

19. Notes for Future Implementation

Ghi chú cho bước sau nếu tôi muốn yêu cầu bạn implement plan này:

Thứ tự nên implement.

Những file nên chỉnh trước.

Những file nên tránh động vào.

Những phần cần backup hoặc review kỹ.

Những thay đổi nên làm thành từng commit nhỏ.

Những phần cần test sau mỗi phase.

Quy tắc quan trọng:

Bắt buộc đọc các Agent Skills, agent instructions, repository guidelines và tài liệu liên quan trước khi lập kế hoạch.

Bắt buộc đọc codebase trước khi đưa ra nhận xét.

Không áp đặt domain hoặc ý tưởng sản phẩm từ bên ngoài.

Không viết code implementation trong bước này.

Không sửa backend.

Không tự ý xoá file.

Không thay đổi API contract.

Không tự ý cài thêm package.

Không tự ý đổi architecture lớn.

Chỉ tạo file Markdown plan.

Khi nhắc đến file trong codebase, hãy ghi rõ đường dẫn file cụ thể.

Nếu chưa chắc chắn điều gì, ghi vào phần Assumptions hoặc Open Questions.

Plan phải đủ chi tiết để một developer khác có thể đọc file Markdown này và triển khai lại UI mobile mà không cần hỏi thêm quá nhiều.

8. **Câu prompt:** bạn sửa lại nội dung của phần landing page và các phần khác theo nội dung của codebase do phiên bản đó nó bị cũ là phần cũ rồi bây giờ là

phần mới rồi

9. **Câu prompt:** Bạn hãy đọc và hiểu toàn bộ codebase hiện tại trước khi sửa code.

Mục tiêu chính:

Cải tiến giao diện UI/UX của hệ thống theo hướng hiện đại, trực quan, user-oriented và phù hợp với nội dung ứng dụng. Trọng tâm là trang chủ: hiển thị danh sách các địa điểm bằng card có hình ảnh, tên địa điểm, mô tả ngắn và hành động rõ ràng. Khi người dùng click vào từng card địa điểm, hệ thống sẽ mở/truy cập trang chi tiết hoặc khu vực thông tin chi tiết của địa điểm đó.

Yêu cầu trước khi sửa:

1. Khảo sát toàn bộ codebase để xác định:

Công nghệ frontend đang dùng là gì.

Cấu trúc thư mục chính.

Component/page nào đang là trang chủ.

Routing hiện tại hoạt động như thế nào.

Dữ liệu địa điểm đang lấy từ đâu: hardcoded, JSON, API, database, mock data hay backend.

Có sẵn trang chi tiết địa điểm chưa, nếu chưa thì cần tạo cách hiển thị phù hợp.

2. Sau khi khảo sát, hãy tự lập kế hoạch ngắn gọn trong code comments hoặc trong phần trả lời trước khi sửa: sẽ sửa file nào, thêm component nào, có ảnh hưởng backend không.

3. Ưu tiên không can thiệp backend nếu không cần thiết. Nếu bắt buộc phải sửa backend/API/data schema thì giải thích lý do rõ ràng và giữ thay đổi nhỏ nhất có thể.

Yêu cầu UI trang chủ:

1. Thêm khu vực hiển thị các địa điểm nổi bật.

2. Mỗi địa điểm hiển thị dưới dạng card gồm:

Hình ảnh địa điểm.

Tên địa điểm.

Mô tả ngắn 1–2 dòng.

Thông tin phụ nếu có: vị trí, thời lượng tham quan, tag/chủ đề, đánh giá, trạng thái nổi bật.

Nút hoặc CTA rõ ràng như “Xem chi tiết”, “Khám phá”, “Bắt đầu tham quan”.

3. Toàn bộ card nên có thể click được, không chỉ riêng nút.

4. Khi click vào card:

Nếu project đã có routing chi tiết thì điều hướng tới trang chi tiết địa điểm.

Nếu chưa có routing chi tiết thì tạo route/page chi tiết phù hợp với kiến trúc hiện tại.

Nếu kiến trúc hiện tại không phù hợp để tạo route mới, có thể dùng modal/drawer/section chi tiết, nhưng phải giải thích lựa chọn.

5. Trang chi tiết địa điểm nên hiển thị:

Ảnh lớn/hero image.

Tên địa điểm.

Mô tả đầy đủ.

Các thông tin quan trọng: địa chỉ, giờ mở cửa, giá vé, điểm nổi bật, gợi ý trải nghiệm, lưu ý cho khách tham quan nếu dữ liệu có sẵn.

Nút quay lại trang chủ hoặc quay lại danh sách địa điểm.

6. Nếu codebase chưa có dữ liệu thật cho hình ảnh, hãy tạo dữ liệu mẫu sạch, dễ thay thế sau này. Có thể dùng ảnh local trong assets/public hoặc URL placeholder ổn định. Không dùng ảnh random dễ hỏng link nếu không cần thiết.

Yêu cầu thiết kế:

1. Giao diện phải mobile-first, responsive tốt trên mobile, tablet và desktop.

2. Card phải có hierarchy rõ:

Ảnh là điểm thu hút đầu tiên.

Tên địa điểm nổi bật.

Mô tả ngắn dễ đọc.

CTA dễ bấm.

3. Bố cục gợi ý:

Mobile: 1 cột.

Tablet: 2 cột.

Desktop: 3 hoặc 4 cột tùy không gian.

4. Style nên hiện đại nhưng không quá “AI-generated industrial UI”.

5. Ưu tiên cảm giác giống ứng dụng du lịch/hướng dẫn viên:

Ấm áp.

Có cảm giác khám phá.

Hình ảnh lớn, khoảng trắng hợp lý.

Typography dễ đọc.

Micro-interaction nhẹ khi hover/tap.

6. Không làm giao diện quá màu mè hoặc phá vỡ style hiện tại của project. Hãy kế thừa design system/theme hiện có nếu có.

Yêu cầu accessibility:

1. Ảnh có alt text có ý nghĩa.

2. Card/nút có trạng thái focus rõ ràng khi dùng bàn phím.

3. Không chỉ dùng màu sắc để truyền đạt ý nghĩa.

4. Text phải đủ tương phản với nền.

5. Click target đủ lớn trên mobile.

6. Nếu dùng modal/drawer thì phải đảm bảo có thể đóng bằng nút rõ ràng và không gây khó chịu khi dùng bàn phím.

Yêu cầu performance:

1. Tối ưu ảnh:

Dùng kích thước ảnh hợp lý.

Lazy load ảnh nếu phù hợp.

Không tải ảnh quá lớn không cần thiết.

2. Không thêm thư viện nặng nếu chỉ để làm card đơn giản.

3. Không làm chậm trang chủ.

4. Không phá vỡ build, routing hoặc state hiện tại.

Yêu cầu kỹ thuật:

1. Giữ code sạch, dễ mở rộng.
2. Tách component nếu hợp lý, ví dụ:
DestinationCard
DestinationGrid
DestinationDetail
3. Nếu có TypeScript thì định nghĩa type/interface rõ ràng cho destination.
4. Nếu có dữ liệu mock thì đặt ở vị trí hợp lý như data/destinations.ts, constants, hoặc file tương đương theo convention của project.
5. Không xóa logic cũ nếu chưa chắc chắn. Nếu thay thế UI cũ, hãy giữ lại chức năng quan trọng đang có.
6. Không sửa lan man các phần không liên quan.
7. Không đổi backend, database, auth, payment hoặc API ngoài phạm vi UI địa điểm nếu không bắt buộc.

Yêu cầu nghiệm thu sau khi sửa:

1. Trang chủ hiển thị được danh sách địa điểm có hình ảnh.
2. Click vào từng địa điểm xem được thông tin chi tiết.
3. UI responsive tốt trên mobile và desktop.
4. Không có lỗi console nghiêm trọng.
5. Build/lint/test chạy được nếu project có script tương ứng.
6. Code mới nhất quán với style hiện tại của codebase.
7. Trả lời lại cho tôi:
Bạn đã đọc được cấu trúc project như thế nào.
Bạn đã sửa/thêm những file nào.
Cách chạy để kiểm tra.
Có phần nào bạn phải giả định vì codebase chưa có dữ liệu hay chưa có route không.

Bạn xem xét do đây là thông tin tổng quát, bạn suy nghĩ theo tư duy thiết kế để chọn lọc và thêm thắt những gì mà bạn cho là cần thiết. Với lại những gì không hiểu phải hỏi tôi. Có cần tôi tự làm được thì hãy để tôi tự làm nhé. Bạn không cần tự làm và tôi là bạn đồng hành với bạn

10. **Câu prompt:** Bạn hãy đọc và hiểu toàn bộ codebase hiện tại trước khi sửa code.

Mục tiêu:

Thêm chức năng Blog cho hệ thống. Blog sẽ gồm 2 nhóm nội dung:

1. Một số bài blog có sẵn/được seed sẵn, lấy cảm hứng hoặc metadata từ các nguồn web đáng tin cậy.
2. Bài blog do người dùng tự chia sẻ.

Chức năng blog phải phù hợp với ứng dụng du lịch/hướng dẫn viên du lịch. Không làm kiểu blog chung chung. Nội dung nên xoay quanh:

Kinh nghiệm tham quan địa điểm.
Lịch trình gợi ý.
Mẹo đi lại, mua vé, giờ mở cửa.
Câu chuyện lịch sử/văn hóa.
Review cá nhân.
Âm thực, góc chụp ảnh, lưu ý khi đi du lịch.

Yêu cầu trước khi code:

1. Khảo sát toàn bộ codebase để xác định:
 - Frontend đang dùng framework gì.
 - Backend có tồn tại không.
 - Có authentication/user system chưa.
 - Có database chưa.
 - Có routing frontend chưa.
 - Có pattern component/page/service/api nào đang được dùng.
 - Có design system/theme/component UI sẵn không.
2. Sau khi khảo sát, lập kế hoạch ngắn:
 - Sẽ thêm route/page nào.
 - Sẽ thêm component nào.
 - Dữ liệu blog lưu ở đâu.
 - Có cần sửa backend/database không.

3. Không sửa lan man ngoài phạm vi chức năng blog.
4. Ưu tiên giữ nguyên kiến trúc hiện tại của project. Nếu project đã có convention rõ ràng thì phải đi theo convention đó.

Phạm vi chức năng cần làm:

A. Trang danh sách blog

Tạo trang blog chính, ví dụ route:

/blog

hoặc route tương đương theo routing hiện tại.

Trang blog cần có:

1. Hero/header ngắn giới thiệu blog du lịch.
2. Danh sách bài viết dạng card.
3. Mỗi card gồm:
 - Ảnh cover.
 - Tiêu đề.
 - Mô tả ngắn/excerpt.
 - Tác giả hoặc nguồn.
 - Ngày đăng.
 - Tag/chủ đề.
 - Thời gian đọc ước lượng.
 - Số lượt thích/comment nếu có.
4. Click vào card để xem chi tiết blog.
5. Có tìm kiếm blog theo tiêu đề/nội dung/tag nếu dễ thêm.
6. Có filter theo tag/chủ đề:
 - Kinh nghiệm
 - Lịch trình
 - Văn hóa - lịch sử
 - Ẩm thực
 - Review
 - Mẹo du lịch
7. Responsive:

Mobile: 1 cột.

Tablet: 2 cột.

Desktop: 3 cột nếu phù hợp.

B. Trang chi tiết blog

Tạo trang chi tiết, ví dụ:

/blog/:slug

hoặc route tương đương.

Trang chi tiết cần có:

1. Ảnh cover lớn.
2. Tiêu đề bài viết.
3. Tác giả/nguồn.
4. Ngày đăng.
5. Tag.
6. Nội dung bài viết.
7. Nút quay lại danh sách blog.
8. Khu vực tương tác:
Like/thả tim.
Bookmark/lưu bài.
Share/copy link.
Comment nếu phù hợp với kiến trúc hiện tại.

9. Nếu bài viết là bài lấy từ nguồn web:

Hiển thị rõ `sourceName`.

Có `externalSourceUrl`.

Không giả vờ nội dung đó là do hệ thống/người dùng tự viết.

Có nút “Đọc bài gốc” hoặc “Nguồn tham khảo”.

10. Nếu chưa có backend thật, tương tác như like/bookmark/comment có thể lưu `localStorage` hoặc `state local` cho bản demo, nhưng phải ghi rõ trong `code/response` rằng đây là `demo persistence`.

C. Bài blog có sẵn từ web/seed data

Tạo seed data ban đầu cho một số bài blog.

Quan trọng:

1. Không copy nguyên văn toàn bộ bài viết từ web nếu không có quyền.
2. Chỉ lưu metadata/tóm tắt/ngữ cảnh:
title
slug
excerpt
coverImage
sourceName
sourceUrl
publishedAt
author nếu có
tags
content dạng ngắn/tự viết lại nếu cần cho demo
isExternal hoặc sourceType
3. Ưu tiên nguồn chính thức hoặc có quyền sử dụng rõ ràng.
4. Nếu không thể truy cập web trong môi trường Codex, hãy tạo data mẫu có cấu trúc rõ ràng và ghi chú vị trí để tôi thay bằng nguồn thật sau.
5. Không dùng link ảnh random dễ chết. Ưu tiên asset local trong public/assets hoặc URL ổn định nếu project đang dùng URL.

D. Người dùng chia sẻ blog riêng
Thêm chức năng tạo bài blog mới.

Tạo route/page hoặc modal:

/blog/new

hoặc cách tương đương theo kiến trúc hiện tại.

Form tạo blog cần có:

1. Tiêu đề.
2. Ảnh cover hoặc URL ảnh cover.
3. Mô tả ngắn.
4. Nội dung bài viết.

5. Tag/chủ đề.

6. Tên tác giả:

Nếu project đã có auth thì dùng user hiện tại.

Nếu chưa có auth thì cho nhập displayName hoặc dùng “Khách”.

7. Nút lưu bản nháp nếu dễ thêm.

8. Nút đăng bài.

Validation:

1. Tiêu đề không được rỗng.

2. Nội dung không được rỗng.

3. Excerpt nên giới hạn độ dài hợp lý.

4. Cover URL nếu có thì phải validate cơ bản.

5. Tag phải thuộc danh sách cho phép hoặc có cơ chế tạo tag hợp lý.

Persistence:

1. Nếu project đã có backend/database:

Thêm model/schema BlogPost theo convention hiện tại.

Thêm API tạo bài, lấy danh sách, lấy chi tiết, like/bookmark/comment nếu phù hợp.

Không phá vỡ API cũ.

2. Nếu chưa có backend/database:

Implement bản demo bằng localStorage hoặc data store frontend.

Blog người dùng tạo sẽ hiển thị lại trong danh sách sau khi tạo.

Ghi rõ đây là local-only demo.

3. Nếu project có auth nhưng chưa có role/moderation:

Bài người dùng có thể có status: draft, pending, published.

Nếu làm moderation quá lớn thì chỉ thiết kế field status và UI đơn giản, không làm phức tạp.

E. Tương tác với blog

Thêm các tương tác cơ bản:

1. Like/unlike.
2. Bookmark/unbookmark.
3. Share/copy link.
4. Comment nếu phù hợp.

Yêu cầu comment:

1. Không cho comment rỗng.
2. Hiển thị tên người comment và thời gian.
3. Nếu chưa có auth thì dùng tên “Khách” hoặc input tên.
4. Phải xử lý/sanitize nội dung người dùng nhập, không render HTML nguy hiểm.
5. Không cho phép script tag, inline event handler hoặc HTML nguy hiểm.

F. UI/UX

Thiết kế blog phải hòa hợp với app du lịch:

1. Card đẹp, giàu hình ảnh.
2. Không gian thoáng.
3. Typography dễ đọc.
4. Mobile-first.
5. CTA rõ ràng:
 - Đọc tiếp
 - Viết bài chia sẻ
 - Lưu bài
 - Chia sẻ
6. Trang chi tiết phải đọc thoải mái, không quá chật.
7. Form viết blog phải thân thiện, không giống form admin khô khan.
8. Có empty state:
 - Không có blog.
 - Không tìm thấy kết quả.
 - Chưa có comment.

9. Có loading/error state nếu có gọi API.

G. Security

Vì có user-generated content, cần xử lý bảo mật:

1. Không render trực tiếp HTML người dùng nhập nếu chưa sanitize.
2. Nếu dùng Markdown editor thì render Markdown an toàn.
3. Escape output hoặc dùng sanitizer phù hợp với stack hiện tại.
4. Không lưu dữ liệu nhạy cảm vào localStorage.
5. Nếu có API, validate input cả frontend và backend.
6. Không tin dữ liệu từ client.

H. Accessibility

1. Ảnh blog có alt text có ý nghĩa.
2. Button có label rõ ràng.
3. Card click được nhưng vẫn dùng semantic link/button đúng.
4. Focus state rõ khi dùng bàn phím.
5. Màu chữ đủ tương phản.
6. Không chỉ dùng màu để thể hiện trạng thái.
7. Form có label đầy đủ.

I. Performance

1. Lazy load ảnh blog nếu phù hợp.
2. Không thêm thư viện nặng nếu không cần.
3. Không tải toàn bộ nội dung blog quá lớn ở trang danh sách nếu có thể chỉ dùng excerpt.
4. Nếu có nhiều bài, thiết kế sẵn khả năng pagination hoặc load more, nhưng chưa cần làm quá phức tạp nếu project nhỏ.

J. Cấu trúc dữ liệu gợi ý

Nếu chưa có model sẵn, có thể dùng cấu trúc tương tự:

BlogPost:

id

slug
title
excerpt
content
coverImage
coverAlt
authorName
authorAvatar optional
sourceType: "seedkern.03em "externalkern.03em "user"
sourceName optional
sourceUrl optional
tags
createdAt
updatedAt
publishedAt
readingTime
likesCount
commentsCount
bookmarksCount optional
status: "draftkern.03em "pendingkern.03em "published"

Comment:

id
postId
authorName
content
createdAt

K. Kết quả mong muốn

Sau khi hoàn thành:

1. Có trang danh sách blog.
2. Có trang chi tiết blog.
3. Có một số bài blog seed sẵn.
4. Người dùng có thể tạo/chia sẻ blog của riêng mình.
5. Người dùng có thể tương tác với blog: like, bookmark, share, comment nếu phù hợp.

6. UI responsive tốt.
7. Không phá chức năng cũ.
8. Không có lỗi console nghiêm trọng.
9. Build/lint/test chạy được nếu project có script.

L. Sau khi sửa xong, hãy trả lời lại tôi:

1. Bạn đã phát hiện codebase dùng stack gì.
2. Bạn đã thêm/sửa những file nào.
3. Bạn đã chọn cách lưu blog như thế nào và vì sao.
4. Cách chạy project để test.
5. Các chức năng blog đã làm được.
6. Các giới hạn còn lại, ví dụ localStorage demo, chưa có moderation thật, chưa có upload ảnh thật, chưa có backend thật.

Lưu ý: Đây là ý kiến cá nhân của tôi và có sự trợ giúp prompt bằng các nguồn AI khác. Bạn hãy tùy biến theo codebase hệ thống. Có gì bạn không hiểu phải hỏi tôi. Nếu tôi có thể tự làm phần nào đó bạn có thể nói cho tôi để tôi tự làm, giống như là lấy data blog có sẵn

11. **Câu prompt:** Bây giờ tôi muốn bạn làm cho nâng cấp
Hệ chiếu tham quan Đại Nội

Người dùng đi tới mỗi điểm sẽ “check-in” bằng GPS hoặc quét ảnh để nhận stamp. Cuối chuyến có bộ sưu tập 17 dấu mộc, tiến độ, điểm đã đi, thời gian đã nghe. Cảm giác rất rõ: app biến chuyến đi thành hành trình có thành tựu.

Nhật ký sau chuyến đi

Sau khi tham quan, app tự tạo “Kỷ niệm chuyến đi Đại Nội”: các điểm đã ghé, ảnh đã chụp, câu hỏi đã hỏi AI, audio đã nghe, bản đồ lộ trình. Có nút xuất ảnh/poster chia sẻ.

Với yêu cầu: Không được hard code

12. **Câu prompt:**

hãy lên kế hoạch chi tiết để cải tiến code hiện tại thành hệ thống đa luồng, đa chức năng hơn, bên cạnh chỉ phần chat với AI sẽ có bản đồ để người dùng có thể nhìn, chọn địa điểm và hỏi AI về các địa điểm đó.

13. Câu prompt:

từ bản kế hoạch đó và codebase hiện tại, mở rộng hơn nữa là từng phần ta dùng kỹ thuật gì, công nghệ gì để

thực hiện, đánh giá tính khả thi của chúng và cho bản báo cáo kế hoạch cuối cùng

14. Câu prompt:

riêng với phần ảnh map sơ đồ tôi cần chuẩn bị những gì để có thể đáp ứng được yêu cầu ở trên, ghi rõ ràng yêu cầu

15. Câu prompt:

được rồi hãy bắt đầu thực hiện theo plan của bạn, trong quá trình làm cần cung cấp gì thì hãy cho tôi biết.

Và hiện tại map đó đang giành cho trường đại học khoa học tự nhiên, đại học quốc gia thành phố Hồ Chí Minh, nên hãy tự xác định tọa độ dựa trên thông đó nhé. hãy thực hiện đi

16. Câu prompt:

dữ liệu ở đây là ở cs2 cơ sở dữ liệu nên hãy cập nhật lại phần data, hiện nay bạn đã mapping vị trí thực

theo gps lên bản đồ để phục vụ tính năng xác định vị trí chưa, nếu chưa hãy lên kế hoạch chi tiết để thực

hiện và tiến hành làm theo kế hoạch, sau đó viết mọi hướng dẫn setup và chạy ra file guiderun.md cho tôi

17. Câu prompt:

bây giờ tôi muốn loại bỏ hoàn toàn cái giao diện chatbot hiện tại, tôi muốn trong trang chính hiện thị các địa

điểm đã được tích hợp (trong đề án này chỉ có 1 điểm là trường dh khtn) sau đó khi bấm vào thì màn hình hiện

song song map của khtn như ảnh trên và phần chatbot kế bên để chat và nói, cung cấp hình ảnh các thứ rồi tương tác. Khi hiển thị map phải có nút cho reset vị trí theo gps và nút chọn vị trí hiện tại trên bản đồ, kế theo đó có phần chọn địa điểm tiếp theo muốn đến, sau đó AI sẽ chỉ đường và tiến hành thuyết minh về địa điểm đó. hãy lên kế hoạch để cải tiến nó như thế đi.

18. Câu prompt:

Hướng dẫn viên: Giới thiệu ngắn gọn và hấp dẫn về Tòa nhà A

vi 01:08 AM

Chào bạn! Tòa nhà A nghe thật thú vị và đầy hứa hẹn! Tuy nhiên, hiện tại tôi chưa có thông tin cụ thể nào về Tòa nhà A trong dữ liệu của mình. Để tôi có thể giúp bạn khám phá, bạn có thể cho tôi biết Tòa nhà A này nằm ở đâu không, ví dụ như trong một trường đại học, một khu di tích hay một thành phố nào đó? Liệu Tòa nhà A có gắn liền với câu chuyện lịch sử đặc biệt nào không nhỉ?

Hữu ích

Chưa đúng

vi 01:08 AM

Tuyệt vời! Nghe bạn nói "bắt đầu di chuyển" là tôi thấy hào hứng lắm rồi! Chúng ta cùng lên đường khám phá những điều thú vị phía trước nhé. Bạn đã sẵn sàng cho hành trình mới chưa? Hãy cho tôi biết điểm đến tiếp theo của chúng ta là gì, hay bạn muốn tìm hiểu điều gì đặc biệt trên đường đi? Tôi rất sẵn lòng chia sẻ những câu chuyện hấp dẫn. Bạn có muốn tôi kể một câu chuyện về vùng đất này không?

hiện tại khi chọn 1 địa điểm, phản hồi đang bị xung đột, cái trên chưa nói xong thì cái dưới hiện ra như trên,

hãy kiểm tra lại code, tìm nguyên nhân cốt lõi rồi lên kế hoạch khắc phục, đồng thời hãy thêm tính năng chỉ đường nữa đi khi bấm vào điểm đó.

19. Câu prompt:

tôi muốn hệ thống bây giờ dùng thông tin của gg maps về tọa độ các tòa nhà

có trong bản đồ và tương tác thực tế theo các tọa độ đó, khi chọn 1 điểm thì hệ thống sẽ dùng tọa độ điểm được chọn và tọa độ hiện tại (có thể là hiện tại theo lựa chọn của người dùng bằng cách nhấn vào chữ kh phải chỉ gps) sau đó đẩy lên gg maps rồi truy xuất tìm đường và hướng dẫn người dùng cách đi đến đấy nữa, có thể là dùng chatbot để đọc các đi và in ra text phản hồi, hệ thống mặc định truy xuất đường đi theo chế độ đi bộ. hãy lên kế hoạch để thực chúng với demo 3 tòa là nhà điều hành, tòa F và nhà thể thao với tọa độ tương ứng trong file toado.md của tôi ở bên.

20. Câu prompt:

cái gg map api key có mất phí kh và sai được nhiều kh

21. Câu prompt:

hãy chuyển qua dùng openstreetmap đi, hãy làm lại kế hoạch chi tiết

22. Câu prompt:

oke hãy thực hiện đi, đồng thời ghi các thứ tôi phải setup vào 1 file setup.md nữa nhé.

23. Câu prompt:

hệ thống hiện tại chỉ xử lý được yêu cầu đầu tiên còn yêu cầu thứ 2 thì thông báo không thể xử lý được yêu cầu lúc này, hãy thử lại. Hãy tìm nguyên nhân cụ thể và lên kế hoạch để khắc phục, đồng thời hiện tại thì vẫn chưa làm tính năng chỉ đường, khi bấm vào 1 điểm và chọn xác nhận đến nơi này thì nên in ra và đọc chỉ đường sau đó khi người dùng bấm tiếp đã đến nơi thì mới giới thiệu về điểm đó. Đồng thời khi bấm xác nhận đã đến nơi thì cập nhật vị trí hiện tại đến tọa độ đó luôn, hãy lên kế hoạch để cải tiến chúng.

24. Câu prompt:

hiện tại tính năng chỉ đường và thực hiện các thao tác như thay đổi vị trí các thứ như trong kế hoạch trước
vẫn chưa được thực hiện, hãy kiểm tra lại vấn đề và tiến hành bổ sung cho đầy đủ theo như kế hoạch trước đó

25. **Câu prompt:** Đọc toàn bộ codebase của tôi sau đó tiến hành lên kế hoạch cho thay đổi và cải tiến sau:

bây giờ tôi muốn update phiên bản code của tôi sang tham quan kinh thành Huế thay vì trường dh khtn với map kinh thành đã có trong file [map.png](file;file:///c%3A/TranNhatTruong_2026/HK1/Tu%20duy%20tinh%20toan/Project) và tọa độ có trong file [toado.md](file;file:///c%3A/TranNhatTruong_2026/HK1/Tu%20duy%20tinh%20toan/toado.md) hãy xác định từng điểm trong ảnh ứng với công trình nào sau đó mapping tọa độ thực tế cái đó vào trong map (kiểm tra cẩn thận xác định chính xác công trình nào ứng với cái nào rồi mới gắn tạo độ đảo bảo các điểm đúng với công trình tương ứng trên ảnh). Bây giờ khi bắt đầu hệ thống tự định vị gps hoặc người dùng chọn vị trí hiện tại (có 2 nút cho người dùng chọn) và tại bất kì thời điểm nào người dùng cũng có thể định dạng lại vị trí hiện tại thông qua gps hoặc chọn vào map. sau đó khi chọn vào 1 địa điểm thì sẽ hiển thị chỉ đường đến đây hoặc giới thiệu về cái này. khi chọn chỉ đường đến đây thì AI gửi tọa độ gps điểm hiện tại và điểm đến thật của chúng được mapping ban đầu cho gg map hoặc openstreet map để nó truy xuất đường đi. sau đó gửi về AI nó sẽ hướng dẫn người dùng đi, đồng thời in ra khung chat đường. đồng thời trong lúc này thì cũng hiện nút đã đến nơi hay chưa, khi bấm đến nơi rồi thì chuyển vị trí hiện tại đến điểm mới và tiến hành hỏi bạn có muốn giới thiệu về cái này không. Còn nếu người dùng chỉ bấm giới thiệu điểm này thì tiến hành giới thiệu. Hãy lên kế hoạch chi tiết để thực hiện chúng

26. **Câu prompt:**

hiện tại code tôi đang gặp 1 vài vấn đề. đầu tiên là phần xác định vị trí thật theo tọa độ thì chưa chính xác, nó đang chỉ loạn xạ trên map, chưa đúng địa danh địa điểm của nó. Phần này tôi đề xuất bạn hướng dẫn tôi cách tự cấu hình bằng tay cho chính xác hơn. Thứ hai là phần giới thiệu, khi chọn giới thiệu điểm này thì nó không được biết thông tin về điểm sẽ giới thiệu để AI có thể thực hiện. thứ ba là phần chỉ đường, hiện nay nó đang không dùng đúng đường đi trên gg map mà chỉ chỉ đi thẳng tới nơi xuyên địa hình. phần này nên chỉnh chế độ trong bên thứ 3 ta sẽ lấy đường đi là chế độ đi bộ sau đó tiến hành chỉ

đường và đồng thời đọc lên chứ kh chỉ hiện thị bên dưới. đồng thời bên dưới sẽ hiện thanh ngang hướng dẫn đi, bấm mở rộng sẽ hiển thị toàn bộ từng bước di chuyển để đến nơi. hãy kiểm tra lại code, tìm nguyên nhân gây các lỗi trên và tiến hành lên kế hoạch theo mô tả của tôi để khắc phục. sau đó để tôi xác nhận

27. Câu prompt:

tiến hành cải tiến code theo như kế hoạch bạn vừa đề ra. cập nhật các file thông số và hướng dẫn cần thiết

28. Câu prompt:

vẫn còn 1 số vấn đề nhưng bây giờ tôi muốn khắc phục cái map trước, bây h bạn hãy cho tôi 1 bản demo riêng chỉ có cái map và phần kéo thả điều chỉnh vị trí, khi tôi chỉnh xong save lại, thì bạn nhớ vị trí đó và lưu lại vào database, sau này thì map sẽ cố định như vậy luôn cho tất cả lần chạy sau và cho mọi người dùng. hãy lên kế hoạch để thực hiện chúng cho tôi xác nhận

29. Câu prompt:

hãy tiến hành thực hiện theo cái đó đi và làm 1 file setuptoado.md để hướng dẫn tôi các bước thực hiện truy cập các thứ

30. Câu prompt:

hiện tại phần thanh ngang trên chứa thông tin Kinh thành Huế (đại nội) ... xem chat đang che khuất 1 phần bên dưới như nút định vị gps. hãy lên kế hoạch thiết kế lại cho không có sự trùng lặp nên nhau này

31. Câu prompt:

hiện tại khi chạy cấu hình để chỉnh lại map thì tôi chưa thấy phần lưu map lại vào database, hãy kiểm tra lại code, lên kế hoạch khắc phục để chạy bản điều chỉnh map thì có nút lưu map hiện tại để lưu lại thông tin của map.

32. Câu prompt:

tôi vừa cập nhật thêm vị trí của ngõ môn vào trong file toado.md, hãy thêm nó vào bản đồ đi

33. Câu prompt:

khi đến được điểm mới, thì giới thiệu điểm thứ 2 sẽ bị thống báo lỗi và trên màn hình sẽ trả về không thể phản hồi lúc này, hãy thử lại sau, hãy kiểm tra lại code và cho biết tại sao

34. Câu prompt:

Xin chào! Hãy tải ảnh hiện vật, dùng webcam hoặc hỏi trực tiếp về một địa điểm lịch sử.

Hữu ích

Chưa đúng

10:10 PM

Hướng dẫn viên: Giới thiệu ngắn gọn và hấp dẫn về Ngọ Môn

10:11 PM

Ngọ Môn: Ngọ Môn là cổng chính phía Nam của Hoàng thành Huế, được xây dựng vào năm 1833 dưới triều vua Minh

Mạng. Cổng có kiến trúc đồ sộ, nguy nga với lầu Ngũ Phụng ở phía trên, nơi diễn ra nhiều sự kiện trọng đại của triều đình nhà Nguyễn như lễ Ban Sóc, lễ Truyền Lô và cũng là nơi vua Bảo Đại tuyên bố thoái vị năm 1945.

Hữu ích

Chưa đúng

10:11 PM

Lộ trình đi bộ từ vị trí của bạn đến Duyệt Thị Đường: Bắt đầu di chuyển từ vị trí hiện tại theo hướng đường Ông

Ích Khiêm khoảng 39 mét. Hãy rẽ trái khoảng 29 mét. Hãy rẽ phải khoảng 74 mét. Hãy rẽ trái vào đường Cửa Quảng

Đức khoảng 35 mét. Hãy rẽ phải vào đường Đường 23 Tháng 8 khoảng 144 mét. Hãy rẽ phải vào đường Đường 23 Tháng

8 khoảng 23 mét. Hãy rẽ trái vào đường Cầu Ngoại Kim Thủy khoảng 67 mét. Hãy rẽ phải khoảng 6 mét. Hãy rẽ trái

khoảng 4 mét. Hãy rẽ phải khoảng 34 mét. Hãy rẽ trái khoảng 170 mét. Hãy rẽ phải vào đường Road 2 khoảng 129

mét. Hãy rẽ trái vào đường Road 9 khoảng 71 mét. Hãy rẽ trái khoảng 23 mét. Hãy rẽ trái khoảng 24 mét. Bạn đã đến nơi.

Hữu ích

Chưa đúng

10:11 PM

Hướng dẫn viên: Giới thiệu ngắn gọn và hấp dẫn về Duyệt Thị Đường

10:11 PM

Ngọ Môn: Ngọ Môn là cổng chính phía Nam của Hoàng thành Huế, được xây dựng vào năm 1833 dưới triều vua Minh

Mạng. Cổng có kiến trúc đồ sộ, nguy nga với lầu Ngũ Phụng ở phía trên, nơi diễn ra nhiều sự kiện trọng đại của triều đình nhà Nguyễn như lễ Ban Sóc, lễ Truyền Lô và cũng là nơi vua Bảo Đại tuyên bố thoái vị năm 1945.

Hữu ích

Chưa đúng

10:11 PM

Lộ trình đi bộ từ vị trí của bạn đến Thế Miếu (Thế Tổ Miếu): Bắt đầu di chuyển từ vị trí hiện tại khoảng 24 mét. Hãy rẽ trái khoảng 226 mét. Hãy rẽ phải khoảng 460 mét. Hãy rẽ trái vào đường Nguyễn Thiện Thuật khoảng 84 mét. Hãy rẽ trái khoảng 42 mét. Bạn đã đến nơi.

Hữu ích

Chưa đúng

10:11 PM

Hướng dẫn viên: Giới thiệu ngắn gọn và hấp dẫn về Thế Miếu (Thế Tổ Miếu)

10:11 PM

Ngọ Môn: Ngọ Môn là cổng chính phía Nam của Hoàng thành Huế, được xây dựng vào năm 1833 dưới triều vua Minh

Mạng. Cổng có kiến trúc đồ sộ, nguy nga với lầu Ngũ Phụng ở phía trên, nơi diễn ra nhiều sự kiện trọng đại của

triều đình nhà Nguyễn như lễ Ban Sóc, lễ Truyền Lô và cũng là nơi vua Bảo Đại tuyên bố thoái vị năm 1945. lần này thì với điểm nào nó cũn thuyết trình về ngọ môn, hãy tìm nguyên nhân gây ra tình trạng này

35. Câu prompt:

đọc lại toàn bộ code base hiện tại, sau đó lên kế hoạch để tiến hành kéo cái bản đồ map của tôi vào đúng vị trí của nó trên bản đồ openstreet maps, cho tôi vài phương án và đề xuất phương án tốt nhất cho tôi xác nhận

36. Câu prompt:

loại bỏ ảnh hiện tại của tôi đi, sau đó tự vẽ lại 1 map đủ các địa điểm trên và vẽ các tòa nhà và biểu tượng vào đúng vị trí của nó để map sinh động hơn, hãy tiến hành lên kế hoạch để thực hiện nó đi

37. Câu prompt:

hãy thêm 1 cái nút nữa, khi bấm vào nó thì map sẽ tự động nhảy tới cổng ngọ môn, tránh trường hợp đi lạc xong tìm map không ra. hay lên kế hoạch thực hiện

38. Câu prompt:

thêm phần là sau khi bấm đi đến đây, trong lúc đang hướng dẫn có 2 option cho người dùng 1 là đã đến và gth 2 là hủy. hãy lên kế hoạch để thực hiện nó

39. Câu prompt:

hãy lên kế hoạch làm thêm tính năng lên lộ trình theo thời gian và số địa điểm, đồng thời có thêm tính năng gợi ý điểm tiếp theo nếu ban đầu người dùng không dùng tính năng gợi ý lộ trình, hãy lên kế hoạch cho cả 2 tính năng cho tôi xác nhận trước

40. Câu prompt:

1. Khi người dùng ở xa thì lấy Ngọ Môn làm điểm xuất phát.
2. Khi giới thiệu xong 1 điểm thì màn hình hiện 1 góc nhỏ bên dưới là gợi ý điểm tiếp theo: tên địa điểm sau đó người dùng bấm khám phá thì chỉ đường

như bth rồi tới nơi mới giới thiệu.

hãy thêm vào kế hoạch

41. Câu prompt:

Tiến hành thực hiện theo kế hoạch của bạn đi

42. Câu prompt:

có 1 điều tôi thấy khá bất tiện là khi bấm vào 1 điểm nó hiển thị lên là chỉ đường đến đây và phần giới thiệu, khi tôi bấm vào chỉ đường đến đây hoặc bấm giới thiệu rồi nó vẫn hiện cái đó trên bản đồ, khiến nhìn chập chờn và rất vướng nên hãy lên kế hoạch để khi đã bấm chỉ đường đến hay giới thiệu rồi thì tự động tắt cái đó trên bản đồ, đồng thời cũng bổ sung thêm khi bấm chỉ đường đến thì có 3 lựa chọn 1 là hủy 2 là đã đến 3 là đã đến và giới thiệu đi, hãy lên kế hoạch cho tôi xác thực trước

43. Câu prompt:

bây giờ tôi muốn có 1 trò chơi cho nhóm bạn chơi cùng nhau khi đi tham quan, bây giờ khi tham quan được vài địa điểm, bạn hãy lấy data của những địa điểm đã tham quan sau đó tạo thành 1 bộ vào câu hỏi sau đó tạo link kahoot cho người dùng quét mã vào chơi chung sau khi trò chơi kết thúc thì nhận data từ kahoot xem ai thắng và khen người đó. tình năng này có khả thi không

44. Câu prompt:

có thể làm để các người khác chỉ cần quét mã bằng camera bình thường mà không cần phải truy cập web cũng có thể chơi được không

45. Câu prompt:

ở chế độ đầu trí nhóm thì máy điện thoại khác có kết nối chung wifi cũng không thể mở được link, hãy kiểm tra và tìm vấn đề sau đó lên kế hoạch khắc phục

46. Câu prompt:

data_huongdanthamquan.txt

từ thông tin hiện có và các thông tin bổ sung trong file trên, hãy tiến hành điều chỉnh prompt để thay đổi cách trả lời. Với hệ thống này là làm để hướng dẫn du lịch nên phải làm cho nó vừa cân bằng được giới thiệu lịch sử và các điểm có thể nhìn tham quan hay xem ở mỗi địa điểm. Nên hãy chỉnh lại để hệ thống phản hồi cân bằng các thông tin: lịch sử của địa điểm, điểm nổi bật, hướng dẫn tham quan. cái hướng dẫn tham quan thì nên kiểu là: bạn có thể nhìn thấy và kế bên đây có cũng đáng xem, nó là minh chứng cho khi di chuyển lên lầu 2 nó có bức trang này hay món đồ này cũng đáng quan tâm, lịch sử của nó ... nó có gì đặc biệt. Kiểu dẫn dắt người tham quan đi tham quan địa điểm vừa chứa khoogn chỉ giới thiệu lịch sử. Thêm nữa bớt đặt câu hỏi cho người dùng ở cuối câu để người dùng tự do tham quan và khám phá và hỏi đáp. Giữ giọng văn cổ truyền nhưng trả lời phong phú từ vựng và hợp lí hơn, hạn chế spam từ. hãy lên kế hoạch chi tiết cho tôi xác nhận

47. **Câu prompt:** bây giờ tôi muốn cho người dùng checkin tại các điểm theo kiểu như photo booth tại mỗi điểm, ở mỗi điểm khi đã đến sẽ có nút nhỏ checkin bên dưới để khi người dùng nhấn vào thì có thể chụp hình với khung nền đã được tạo sẵn, sau đó lưu vào 1 tệp sau này khi kết thúc chuyến đi thì sẽ trả ra 1 album chứa các ảnh đã check in, và khi kết thúc chuyến đi thì chụp tấm cuối để làm ảnh bìa tổng thể, với phần khung ảnh thì lấy trong "c:\TranNhatTruong_2026\HK1\Tu duy tinh toan\Project\main\AI_Tour" đặt tên tương ứng với từng địa điểm, với các địa điểm chưa có ảnh khung khóa thì tạm thời khóa nó đi và sau này bổ sung sau, với mỗi ảnh trong đó sẽ có 1 phần khung ở giữa là màu đen tối, hãy thay phần đen tối đó bằng phần cam của người dùng, còn phần viền xung quanh giữ nguyên để người dùng chụp thì phần cam hiện tại sẽ thay thế phần khung đen đó, đồng thời viết ra prompt mẫu để lần sau tôi bổ sung ảnh về các địa điểm đó thì dùng prompt đó để bạn hiểu bạn cần lọc khung đen như thế nào, thay cam người dùng vào thế nào,... hãy tiến hành lên kế hoạch chi tiết để thực hiện và cho tôi xác nhận trước khi thực hiện

48. **Câu prompt:** với format ảnh như trên, hãy viết 1 prompt để xây dựng 1 cái tương tự cho địa điểm khác dùng các ảnh và nội dung của địa điểm mới, đúng phân bố và đảm bảo đúng và đúng độ to của phần đen ở giữa, phần đen đó không được chen ảnh hay bất kì gì vào. Thêm nữa, yêu cầu chữ phải rõ và sắc

nét. hãy cho tôi prompt chuẩn (đính kèm ảnh mẫu)

49. **Câu prompt:** thêm vào là tôi chỉ cần nhắc tới địa điểm còn những thứ khác nó tự tra cứu để thêm vào
50. **Câu prompt:** điều chỉnh thêm là phần khung đen sẽ nằm ở giữa và chiếm 1/3 chiều dài và full chiều ngang
51. **Câu prompt:** bây giờ tôi muốn người chơi tạo game khi đang tham quan cũng có thể chơi với tư cách chủ phòng. Khi bấm vào game thì nó sẽ hiển thị nhập tên trước rồi mới ra mã phòng để người khác quét tham gia, người chủ phòng có thể bấm bắt đầu chơi và tiếp tục qua câu hỏi, khi mọi người đã trả lời xong thì hãy tự động qua màn tiếp theo. hãy lên kế hoạch chi tiết cho tôi xác minh.

14. Conclusion and Future Work

14.1. Tổng kết các kết quả đạt được

[Người viết: Trần Nhật Trường - 24120486]

Đồ án "AI Tour Guide" đã chứng minh tính khả thi và hiệu quả của việc ứng dụng Trí tuệ Nhân tạo (AI) vào việc nâng tầm trải nghiệm du lịch văn hóa. Vượt qua những giới hạn của một chatbot thông thường, nhóm đã đạt được những kết quả nổi bật sau:

- **Kiến trúc Đa phương thức (Multimodal):** Xây dựng thành công một ứng dụng web hoạt động mượt mà, cho phép du khách tương tác linh hoạt qua văn bản, giọng nói và hình ảnh theo thời gian thực.
- **Triển khai RAG chuẩn xác:** Thiết lập hiệu quả luồng Retrieval-Augmented Generation (RAG) kết hợp giữa PostgreSQL và extension `pgvector`. Giải pháp này đã khắc phục triệt để tình trạng "ảo giác" (hallucination) của các Mô hình Ngôn ngữ Lớn, đảm bảo AI chỉ cung cấp các thông tin lịch sử đã được kiểm chứng.
- **Số hóa Dữ liệu Di sản:** Thiết kế và xây dựng thành công bộ cơ sở dữ liệu không gian - tri thức liên kết cho khu vực Đại Nội Huế, đồng bộ hóa chặt chẽ giữa tọa độ địa lý, thông tin di tích và bối cảnh lịch sử.

- **Hạ tầng Backend bền bỉ:** Phát triển cơ chế dự phòng (Fallback Mechanism) thông minh. Hệ thống có khả năng tự động chuyển đổi mô hình LLM (từ Gemini sang Llama-3) khi xảy ra sự cố API, đảm bảo tính sẵn sàng cao (High Availability).

Tổng thể, hệ thống đã mang lại một trải nghiệm tham quan cá nhân hóa, thông minh và trực quan cho du khách, dù trải nghiệm thực tế tại không gian di tích hay khám phá trực tuyến.

14.2. Mặt hạn chế của hệ thống

[Người viết: Trần Nhật Trường - 24120486]

Mặc dù hệ thống đã hoạt động ổn định và đáp ứng các mục tiêu đề ra, nhóm nhận thấy kiến trúc hiện tại vẫn còn một số điểm cần khắc phục:

- **Độ nhiễu của Vision AI:** Khả năng nhận diện hình ảnh hiện tại phụ thuộc khá nhiều vào điều kiện ánh sáng và góc chụp. Đặc biệt, các công trình kiến trúc cổ trong Đại Nội có sự tương đồng lớn về thiết kế, đôi khi dẫn đến sai số nhận diện nếu thiếu dữ liệu tọa độ GPS bổ trợ.
- **Giới hạn của Cơ sở dữ liệu:** Do tri thức lịch sử được thu thập và chuẩn hóa thủ công, quy mô dữ liệu vẫn còn hữu hạn. Đối với những truy vấn quá chuyên sâu hoặc nằm ngoài phạm vi dữ liệu đã nạp, hệ thống buộc phải từ chối trả lời để tuân thủ nguyên tắc chống ảo giác.
- **Phụ thuộc vào đường truyền mạng:** Hệ thống đòi hỏi kết nối Internet (4G/LTE) liên tục do các tác vụ LLM và chuyển đổi giọng nói (STT) đều được xử lý trên Cloud. Tại một số khu vực khuất sóng, trải nghiệm của du khách có thể bị gián đoạn.
- **Hạn chế về Quản lý Trạng thái (State Management):** Dữ liệu phòng chơi Game (Room) hiện đang được lưu in-memory trên backend, nên sẽ bị mất hoàn toàn nếu khởi động lại server. Tương tự, dữ liệu Passport và Photo Booth được lưu ở `sessionStorage` của trình duyệt, dẫn đến việc không thể đồng bộ trải nghiệm nếu người dùng đổi thiết bị (từ điện thoại sang máy tính).
- **Hạn chế về Triển khai Frontend:** Việc điều hướng trên Frontend hiện tại được xử lý thủ công thông qua path/query thay vì một framework routing hoàn chỉnh (như React Router). Ngoài ra, file triển khai `docker-compose.yml` hiện

tại chỉ mới đóng gói Backend và Database, trong khi Frontend vẫn cần chạy thủ công ngoài môi trường Node.js.

14.3. Hướng phát triển trong tương lai

[Người viết: Trần Nhật Trường - 24120486]

Dựa trên nền tảng mã nguồn (codebase) linh hoạt đã xây dựng, nhóm đề xuất các hướng phát triển chiến lược nhằm hoàn thiện và thương mại hóa dự án trong tương lai:

- **Triển khai Đám mây (Cloud Deployment) diện rộng:** Mục tiêu ưu tiên hàng đầu là chính thức đóng gói và triển khai ứng dụng lên các nền tảng đám mây (như AWS, Google Cloud, Vercel). Việc này giúp hệ thống vượt ra khỏi môi trường thử nghiệm cục bộ, tiếp cận được lượng lớn du khách thực tế và có thể đo lường tải trọng hệ thống.
- **Mở rộng quy mô Dữ liệu Không gian:** Không chỉ dừng lại ở Kinh thành Huế, nhóm sẽ tiến hành thu thập, số hóa và tích hợp dữ liệu của các khu vực di tích, danh lam thắng cảnh khác trên toàn quốc (ví dụ: Phố cổ Hội An, Tràng An, Thánh địa Mỹ Sơn) để hệ thống trở thành một nền tảng du lịch quốc gia.

15. Phụ lục

15.1. Phụ lục A: Danh sách API

[Người viết: Trần Nhật Trường - 24120486]

Nhóm tính năng	Endpoint hiện có
Health & Metrics	GET /api/v1/health, /live, /ready, /ai, GET /api/v1/metrics
Vision	POST /api/v1/recognize
Chat & TTS	POST /api/v1/chat/unified, GET /api/v1/tts/fetch
Voice (Legacy)	POST /api/v1/voice/chat, POST /api/v1/voice/chat/stream
Feedback	POST /api/v1/feedback
Map & Tour	GET /api/v1/map/route, GET/POST /api/v1/map/config, GET /api/v1/map/plan-tour, GET /api/v1/map/next-suggestion
Quiz Game	POST /api/v1/game/create, /join, /start, /answer, /next, /end, GET /api/v1/game/room/{room_code}/status, /praise, /local-ip
Blog	GET/POST /api/v1/blog-posts, GET /api/v1/blog-posts/{slug}, POST /comments, POST /interactions
Payment (VNPay)	GET /api/v1/payment/food-items, POST /api/v1/payment/create, GET /return, GET /ipn, GET /info/{order_code}
Rating	POST /api/v1/ratings, GET /api/v1/ratings/location/{location_id}, GET /api/v1/ratings/location/{location_id}/summary

15.2. Phụ lục B: Cấu trúc Database

[Người viết: Trần Nhật Trường - 24120486]

Hệ thống PostgreSQL bao gồm các bảng được thiết kế liên kết chặt chẽ:

- **Cốt lõi & Di tích:** locations, artifacts, bilingual_content, knowledge_facts.
- **Tìm kiếm Vector & Không gian:** artifact_faqs (lưu Vector(768)), artifact_relations.
- **Tương tác & Trạng thái:** chat_turns, feedback_events, precomputed_audio.
- **Cộng đồng & Giao dịch:** blog_posts, blog_comments, location_ratings, payments.

15.3. Phụ lục C: Cấu hình yêu cầu

[Người viết: Trần Nhật Trường - 24120486]

Phía Server (Cấu hình tối thiểu để chạy):

- CPU: 4 Core (x86_64 hay ARM64 gì cũng được)
- RAM: 8 GB (ít quá nó giật)
- Ổ cứng: 50 GB SSD cho lẹ
- Hệ điều hành: Ubuntu 22.04 LTS hoặc Linux tương đương
- Phần mềm: Có cài Docker Engine 24.0+ và Docker Compose v2+

Phía Khách xài app (Client):

- Máy: Smartphone mới mới tí hoặc máy tính.
- Trình duyệt: Chro

15.4. Phụ lục A: Danh sách API

me 90+, Safari 14+, Firefox 90+ (Bắt buộc phải hỗ trợ đọc giọng nói Web Speech API và bản đồ WebGL).

- Mạng: 4G hoặc Wi-Fi phải mạnh thì AI mới phản hồi lẹ được.